

Service-oriented Software Engineering (SoSE)

My First SOAP Web service

Prof. Marco Autili

Department of Information Engineering Computer Science and Mathematics

University of L'Aquila

marco.autili@univaq.it

<http://people.disim.univaq.it/marco.autili/>

What Java version should we use?

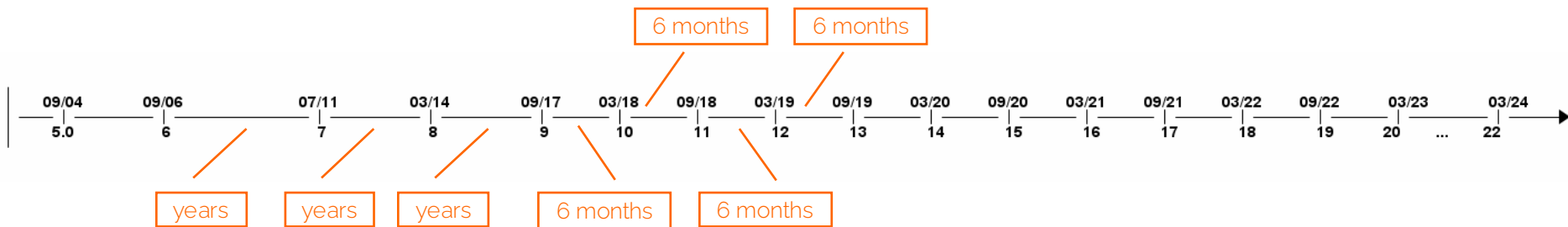


Although this course **is not** about the Java language, before going ahead, let us have a brief discussion on Java releases so to answer the question:

what Java version should we use?

What Java version should we use?

- The last Long-term Support version (LTS) of Java is Java 25, released on September 16, 2025
- On March 17, 2026, Java 26 is about to follow
- Newer Java versions now follow every 6 months
- In the past, Java release cycles were much longer, up to 3-5 years



- The older, longer release-cycles (3-5 years, up until Java 8) meant a lot of new features per release
- The 6-month release cycle means a lot less features, per release

Back to the discussion on rapid technological change... →

https://en.wikipedia.org/wiki/Java_version_history
<https://www.marcobehler.com/guides/a-guide-to-java-versions-and-features>

Why are many companies still stuck with Java 8?

- Build tools (Maven, Gradle, etc.) and some libraries initially had bugs with **Java versions > 8** and needed updates
- Up until Java 8 you were pretty much using Oracle's JDK builds, and you did not have to care about licensing. Oracle changed the licensing scheme in 2019, though, which led the internet go crazy with a ton of articles saying "Java is not free anymore" - and a fair amount of confusion followed. This is however not really an issue... "it is enough" to read about Java distributions and licensing
- Some companies have policies to only use LTS versions and rely on their OS vendors to provide them these builds, which takes time

Summing up

There are a mix of **practical issues** (upgrading your tools, libraries, frameworks) and "**internal company policies**"

<https://www.marcobehler.com/guides/a-guide-to-java-versions-and-features>

Java backwards compatibility

- The good news, however, is that Java is extremely backwards compatible
- This means that programs written with previous version of Java are guaranteed to run with newer version of Java virtual machines - with a few exceptions you do not need to worry about for this course
- The other way around obviously does not work, because your program may use latest features that are simply not available under previous versions

Ingredients

Java SE Development Kit 8

<https://www.oracle.com/java/technologies/javase/javase-jdk8-downloads.html>

Eclipse IDE for Enterprise Java Developers

<https://www.eclipse.org/downloads/packages>

We will start our practical sessions by showing

"How we did it in the past to understand how it is done in the present" ☺

Then, in a future lecture, I will show how to realize the very same service we are going to realize here by using Java 21 and Jakarta EE

(Jakarta EE is the evolution of Java EE after the platform was transferred from Oracle to the Eclipse Foundation in 2017, and was officially launched in 2019)



Download your Java SE Development Kit 8

ORACLE

Products Industries Resources Customers Partners Developers Events



View Accounts



Java downloads

Tools and resources

Java archive

Java SE Development Kit 8u321

Java SE subscribers will receive JDK 8 updates until at least December of 2030.

The Oracle JDK 8 license changed in April 2019

The [Oracle Technology Network License Agreement for Oracle Java SE](#) is substantially different from prior Oracle JDK 8 licenses. This license permits certain uses, such as personal use and development use, at no cost -- but other uses authorized under prior Oracle JDK licenses may no longer be available. Please review the terms carefully before downloading and using this product. FAQs are available [here](#).

Commercial license and support are available for a low cost with [Java SE Subscription](#).

JDK 8 software is licensed under the [Oracle Technology Network License Agreement for Oracle Java SE](#).

JDK 8u321 [checksum](#)

Linux macOS Solaris Windows

Product/file description	File size	Download
x64 DMG Installer	198.54 MB	jdk-8u321-macosx-x64.dmg

Don't have an Oracle Account?

Create Account

© Oracle | [Terms of Use](#) | [Privacy Policy](#)

Download the Eclipse Installer

The Eclipse Installer 2024-03 R now includes a JRE for macOS, Windows and Linux.

Try the Eclipse **Installer** 2024-03 R

The easiest way to install and update your Eclipse Development Environment.

[Find out more](#)

📄 **27,511 Installer Downloads**

📄 **21,426 Package Downloads and Updates**

Download

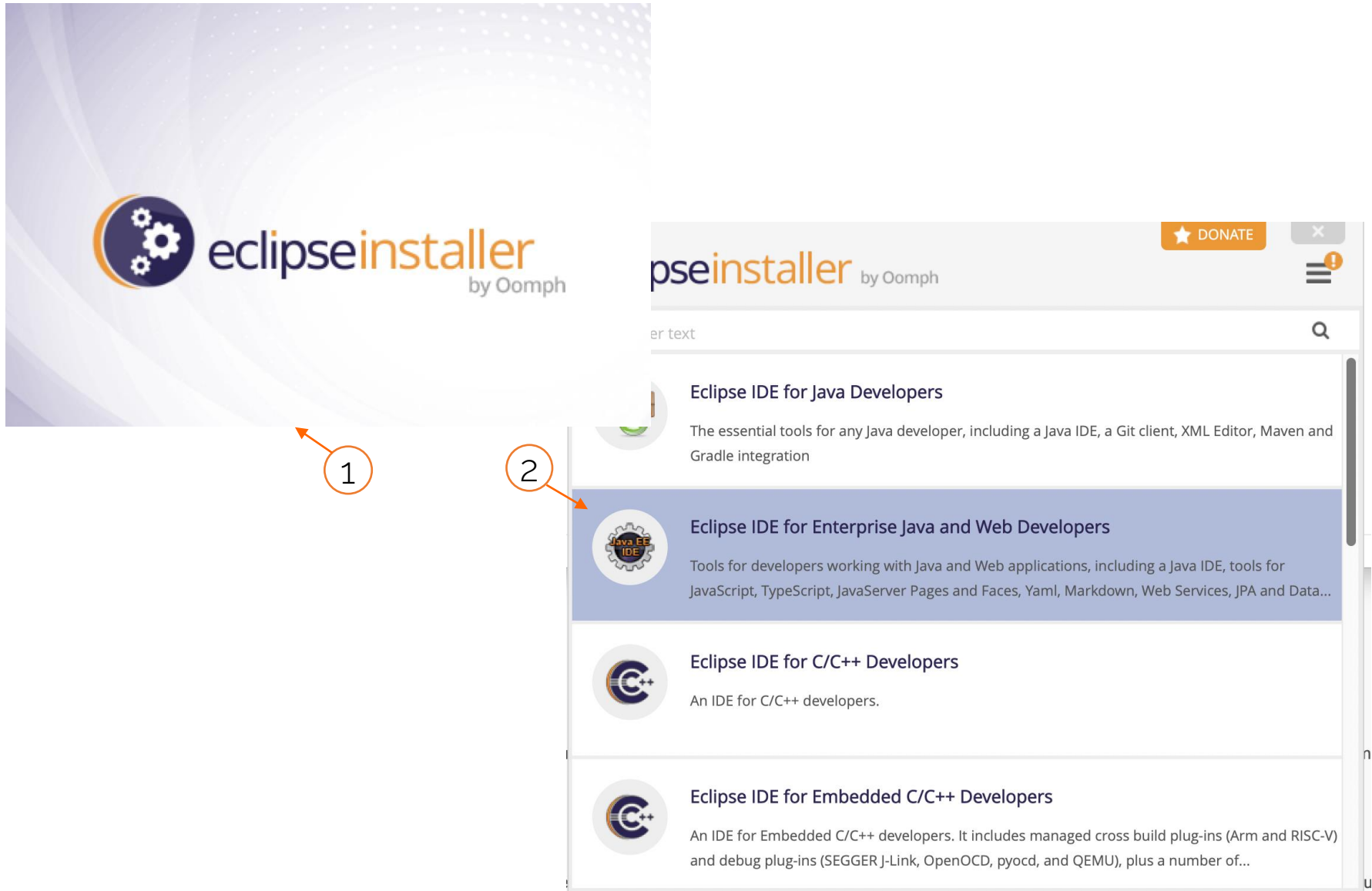
macOS [x86_64](#) | [AArch64](#)

Windows [x86_64](#)

Linux [x86_64](#) | [AArch64](#)

Eclipse IDE 2024-03 R Packages

Download the IDE for Enterprise Java Developers



The image shows a screenshot of the Eclipse Installer website. On the left, there is a large banner with the Eclipse Installer logo and the text "eclipseinstaller by Oomph". An orange arrow labeled "1" points from this banner to the right. On the right, there is a sidebar with a search bar and a list of IDE options. An orange arrow labeled "2" points from the "Eclipse IDE for Enterprise Java and Web Developers" option in the sidebar to the main content area. The main content area displays the details for the selected IDE.

eclipseinstaller by Oomph

er text

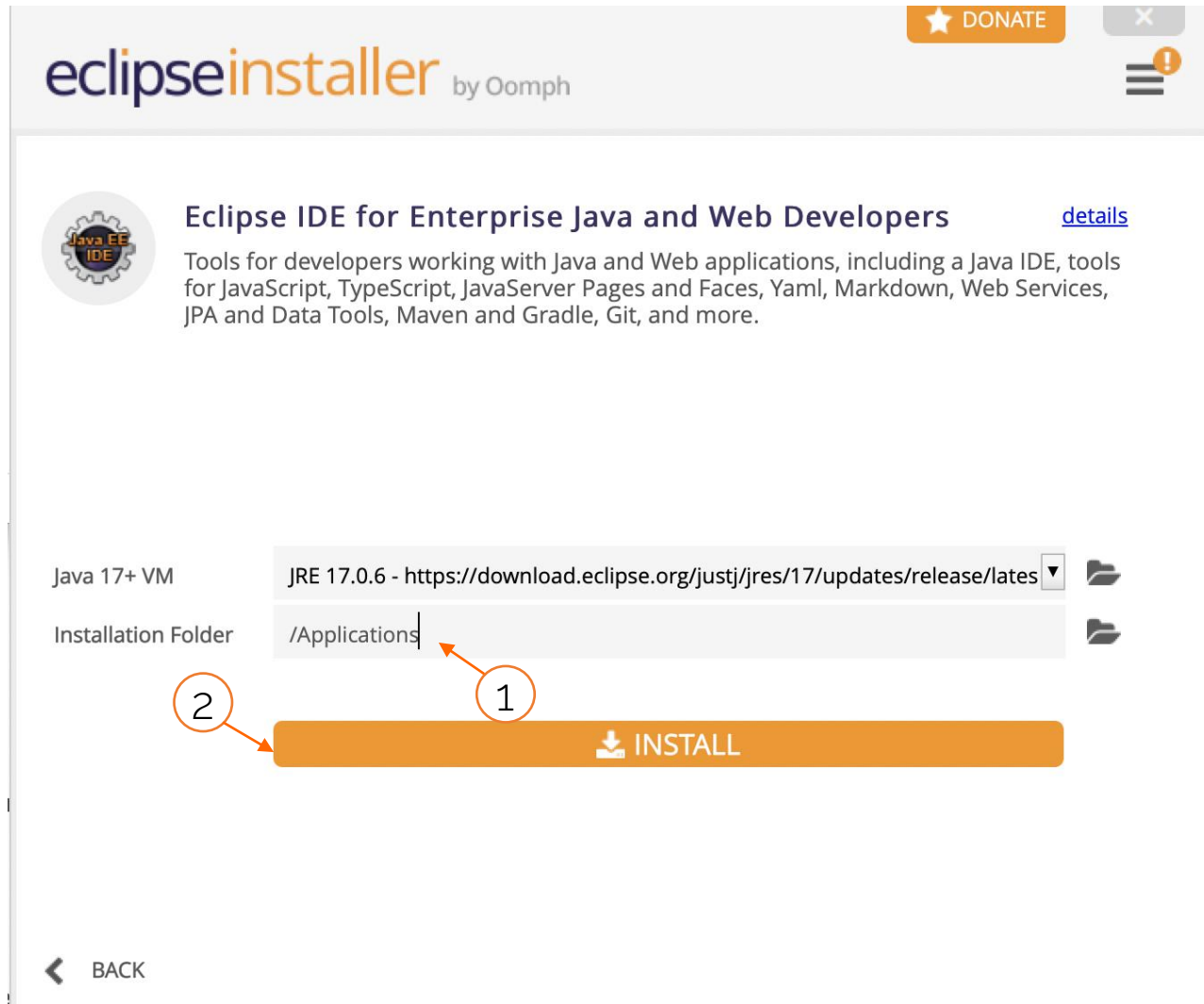
Eclipse IDE for Java Developers
The essential tools for any Java developer, including a Java IDE, a Git client, XML Editor, Maven and Gradle integration

Eclipse IDE for Enterprise Java and Web Developers
Tools for developers working with Java and Web applications, including a Java IDE, tools for JavaScript, TypeScript, JavaServer Pages and Faces, YAML, Markdown, Web Services, JPA and Data...

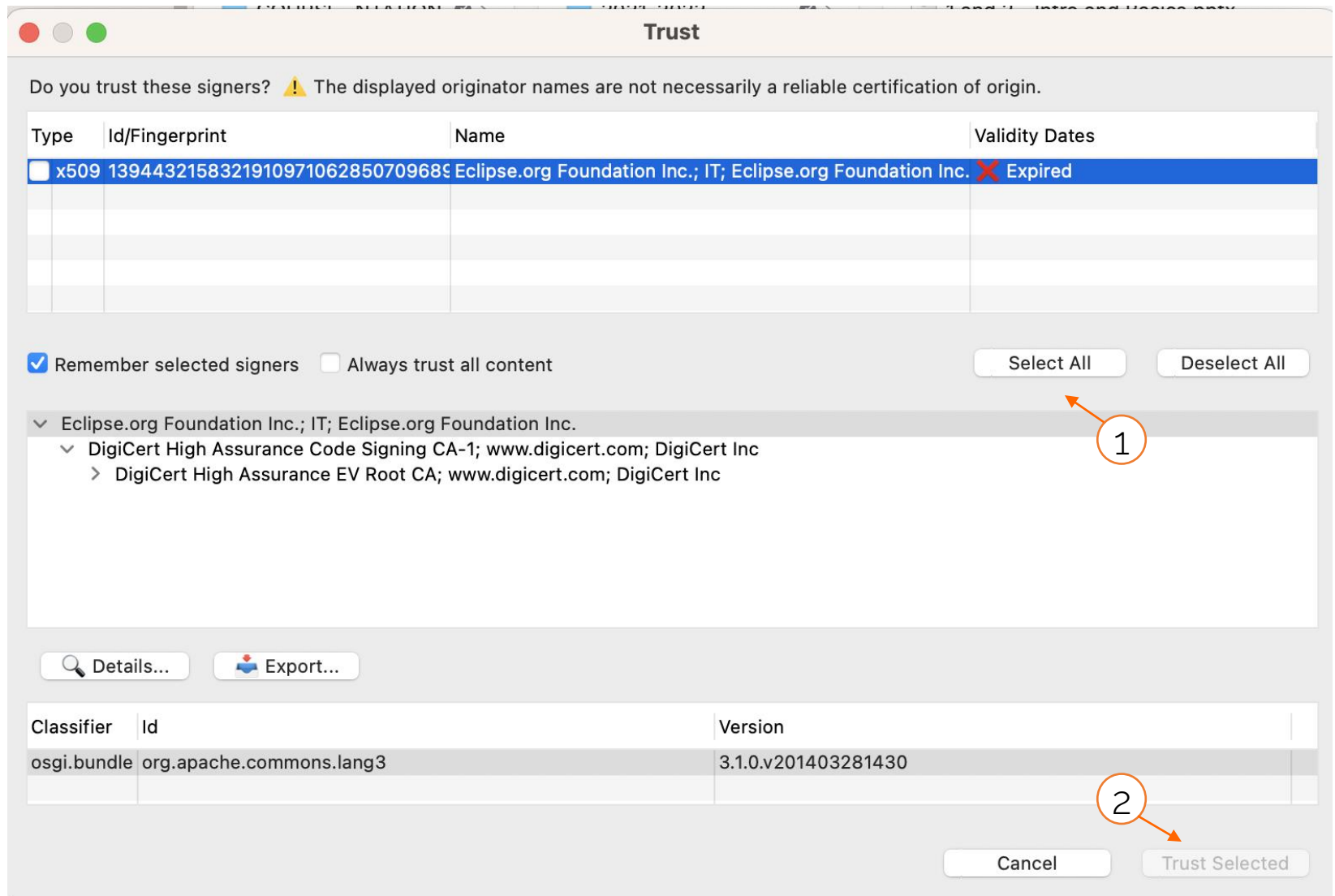
Eclipse IDE for C/C++ Developers
An IDE for C/C++ developers.

Eclipse IDE for Embedded C/C++ Developers
An IDE for Embedded C/C++ developers. It includes managed cross build plug-ins (Arm and RISC-V) and debug plug-ins (SEGGER J-Link, OpenOCD, pyocd, and QEMU), plus a number of...

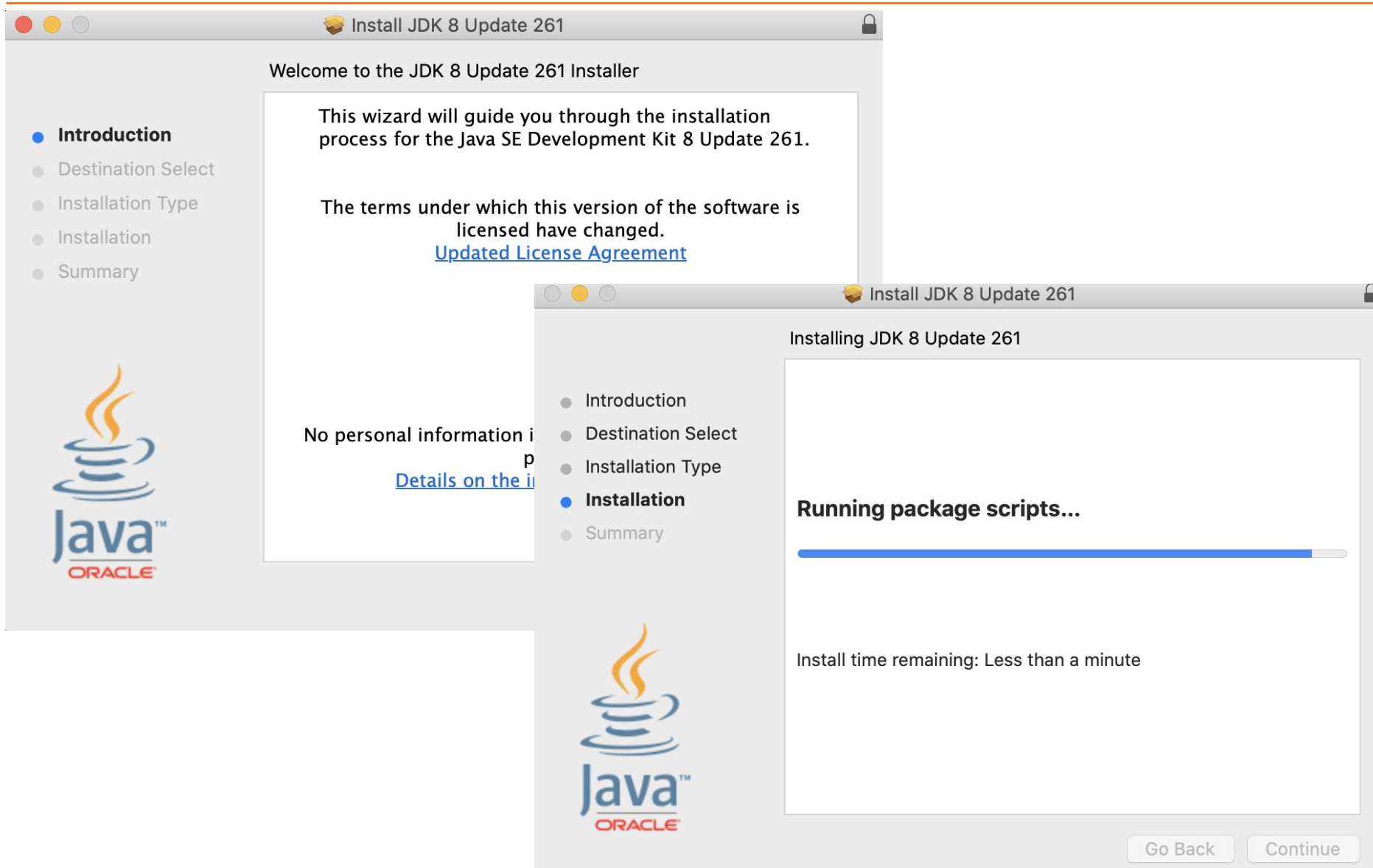
Install the Eclipse IDE for Enterprise Java Developers



Install the Eclipse IDE for Enterprise Java Developers



Install the Java SE Development Kit 8 on MAC



Few slides on the JAVA_HOME

A number of additional slides on the **JAVA_HOME** follow

Indeed, these slides should not be part of this presentation... However, I decided to keep them here for the convenience of those interested students

Not interested students can jump directly to the slide
"My first SOAP WS" and launch Eclipse 😊

Setting JAVA_HOME on MacOS

For the current terminal session only

Type the following command on the terminal every time you need to set JAVA_HOME

- `export JAVA_HOME=/Library/Java/JavaVirtualMachines/jdk1.8.0_66.jdk/Contents/Home`

Permanently for a single user or globally for all users

Depending on the MacOS version and the default shell, add the above command to

- `~/ .zprofile`
- `~/ .bash_profile`
- `~/ .profile`

Setting JAVA_HOME on my Mac (from previous lecture)

- I created `~/.zprofile` (i.e., `/Users/marcoautili/.zprofile`) since user accounts in MacOS Catalina and later Mac OS default to using `zsh`, and not `bash` anymore

- After updating to Catalina, I do not have the file `~/.bash_profile` anymore

- Open up Terminal
- Run the following command



`sudo nano ~/.zprofile`

Create `~/.zprofile` if it does not exist already (e.g., `touch ~/.zprofile`)

- Enter your password, when prompted
- Go to the bottom of the file, and enter the path you wish to add
- Hit `control-x` to quit
- Enter `"Y"` to save the modified buffer

`export JAVA_HOME=/Library/Java/JavaVirtualMachines/jdk-21jdk/Contents/Home`

IMPORTANT: for the file `.zprofile` to be read each time you open the terminal, make sure that, for the current user account, `zsh` is the default shell. To update your account to **use `zsh` by default**:

`chsh -s /bin/zsh`

```
marcoautili@iMac-di-User ~ %  
marcoautili@iMac-di-User ~ % cat ~/.zprofile  
export JAVA_HOME=/Library/Java/JavaVirtualMachines/jdk-21.jdk/Contents/Home  
marcoautili@iMac-di-User ~ %
```



Setting JAVA_HOME on my Mac

```
marcoautili — less · man java_home — 100x39

NAME
    java_home - return a value for $JAVA_HOME

SYNOPSIS
    /usr/libexec/java_home [options]

DESCRIPTION
    The java_home command returns a path suitable for setting the JAVA_HOME environment variable. It determines this path from the user's preferred JVM based on configured environment variables. Additional constraints may be provided to filter the list of JVMs available. By default, if no constraints match the available list of JVMs, the default order is used. The path is printed to standard output.

OPTIONS
    -v or --version version
        Filters the returned JVMs by version (as if JAVA_VERSION had been set in the environment). Example versions: "11.0", or "14.0.2".

    -a or --arch architecture
        Filters the returned JVMs by the architecture they support (as if JAVA_ARCH had been set in the environment). Example architectures: "x86_64", or "arm64".

    -F or --failfast
        Immediately fails when filters return no JVMs; does not print out the path to the default $JAVA_HOME.

    --exec command ...
        Executes the command at $JAVA_HOME/bin/<command> and passes the remaining arguments. Any arguments to select which $JAVA_HOME to use must precede the --exec option.

    -X or --xml
        Prints a list of all matching JVMs and associated properties as an XML plist to stdout.

    -V or --verbose
        Prints a list of all matching JVMs and architectures to stderr.
```

From OS X 10.5
Leopard (released
on October 26,
2007), Apple
introduced
a command line
tool `/usr/libexec`
`/java_home`

Two possible ways to (not the only ones)

Use the following command to **set the JAVA_HOME variable explicitly** with the path of the specific **JDK** you want to use for **developing purpose**, e.g.,

```
export JAVA_HOME=/Library/Java/JavaVirtualMachines/jdk1.8.0_271.jdk/Contents/Home
```

OR

Use the `-v` option

```
export JAVA_HOME=$(/usr/libexec/java_home -v "1.8.0")
```

```
marcoautili@iMac ~ %  
marcoautili@iMac ~ % cat .zprofile  
# export JAVA_HOME=$( /usr/libexec/java_home )  
export JAVA_HOME=/Library/Java/JavaVirtualMachines/jdk1.8.0_271.jdk/Contents/Home  
# export JAVA_HOME=$( /usr/libexec/java_home -v "1.8.0" )  
marcoautili@iMac ~ %  
marcoautili@iMac ~ % █
```

Rapid technological changes: summing up

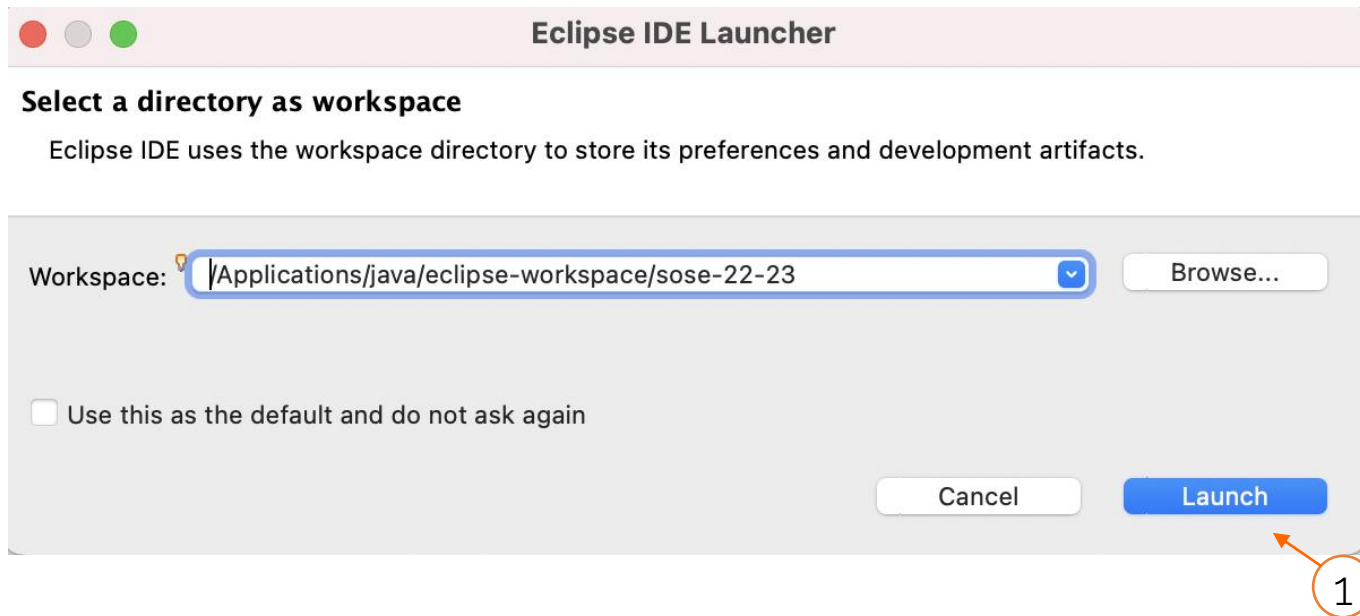
The never-ending story

- The content of some of the coming slides may “not precisely apply” as described because of Operating System updates, Java updates, or Eclipse updates
- Depending on our “system configuration,” you may have (slightly) different cases...
- For example, depending on your Eclipse version, the coming slides might appear slightly different... 😊
- Moreover, you will see **Microsoft Window-based screenshots** interleaving with **macOS-based ones**... 😊

My first SOAP WS

Now, after that brief digression on JAVA_HOME, let's return to the main topic

Launching Eclipse...



Choosing the right JRE

Eclipse File Edit Settings... Services Hide Eclipse Hide Others Show All Quit Eclipse

Preferences

The selected JRE does not support the current compiler compliance level of 17

Add, remove or edit JRE definitions. By default, the checked JRE is added to the build path of newly created Java projects.

Installed JREs:

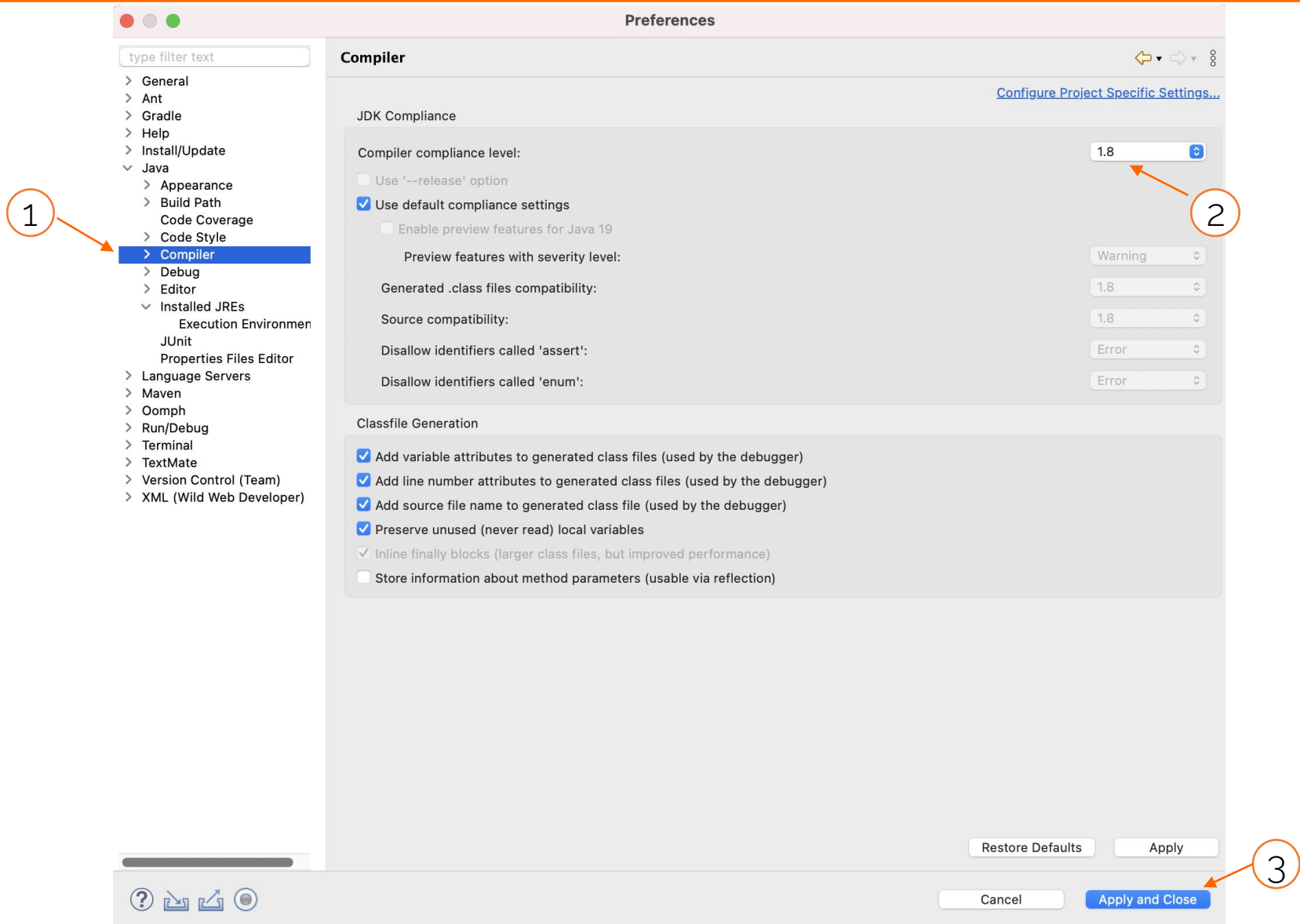
Name	Location
☒ Java SE 8 [1.8.0_66]	/Library/Java/JavaVirtualMachines/jdk1.8.0_66.jdk/Contents/Home
☐ JRE [17.0.6]	/Users/marcoautili/.p2/pool/plugins/org.eclipse.justj.openjdk.hotspot.jre.full.macosx.x86_64_17.0.6.jre/Contents/Home

Annotations:

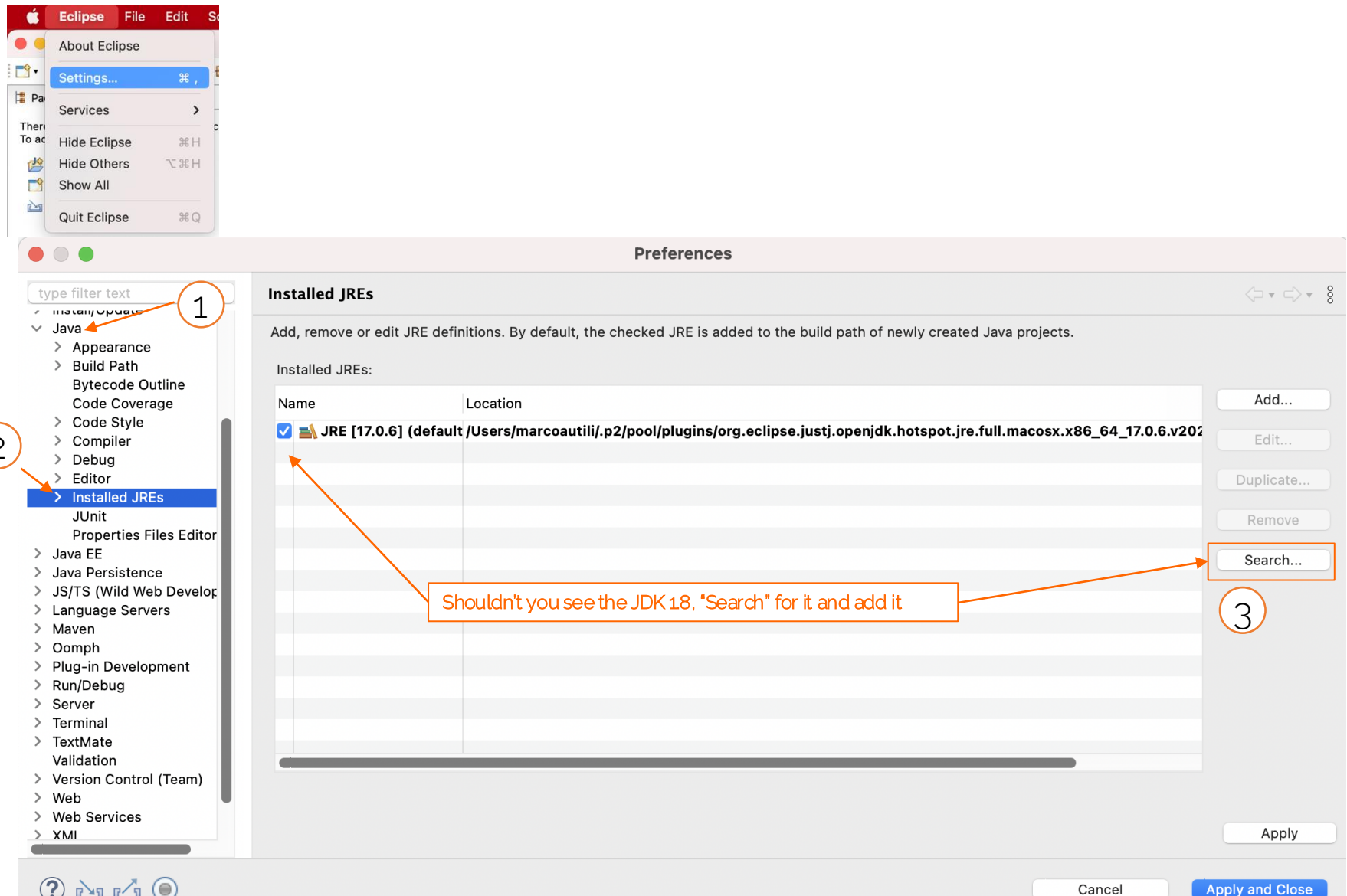
- 1: Java
- 2: Installed JREs
- 3: Java SE 8 [1.8.0_66]

Buttons: Add... Edit... Duplicate... Remove Search...

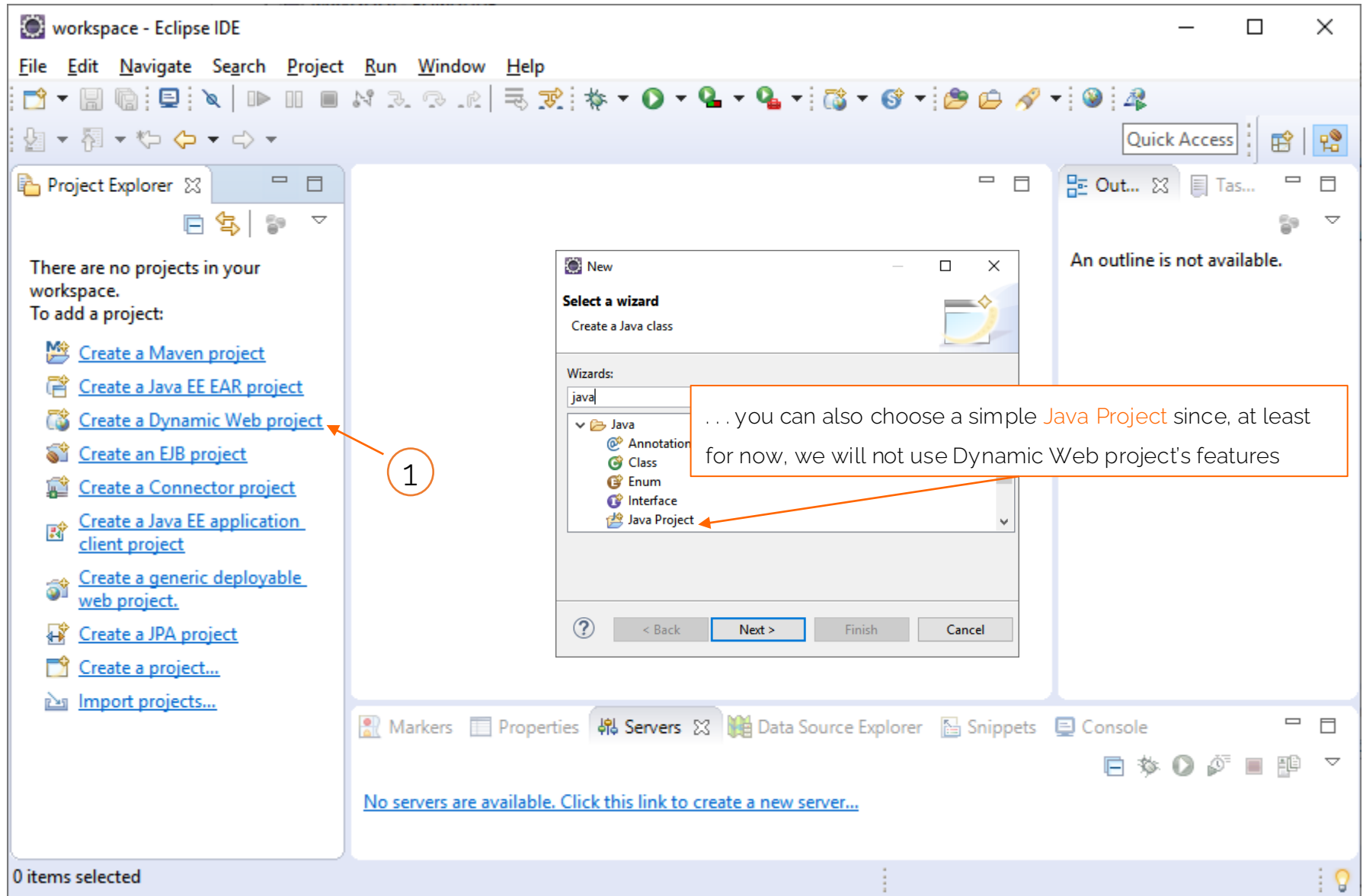
Choosing the right compiler compliance level



Adding the desired JRE



New Dynamic Web Project



New Dynamic Web Project

workspace - Eclipse IDE

File Edit Navigate Search

Project Explorer

There are no projects in your workspace.
To add a project:

Dynamic Web Module version correlates with Servlet API version

For now, all you need to know is that a Servlet is an object that receives a request and send a response back based on that request

We now have up to 6.1

New Dynamic Web Project

Dynamic Web Project

Create a standalone Dynamic Web project or add it to a new or existing Enterprise Application.

Project name: SimpleSumSOAPService

Project location

☒ Use default location

Location: C:\sose-example\workspace\SimpleSumSOAPService

Target runtime

<None>

Dynamic web module version

3.0

Configuration

Default Configuration

The default configuration provides a good starting point. Additional facets can later be installed to add new functionality to the project.

EAR membership

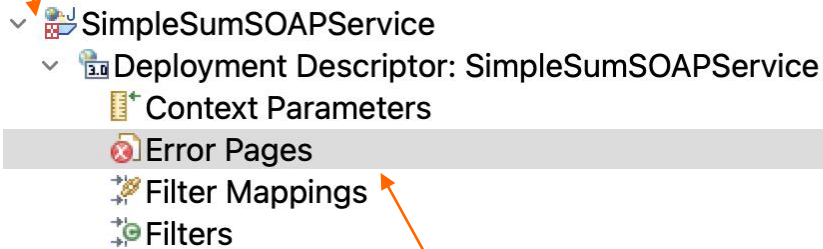
< Back Next > Finish Cancel

Servlet API history

Servlet API history				
Servlet API version	Released	Specification	Platform	Important Changes
Jakarta Servlet 6.1	March 28, 2024	6.1	Jakarta EE 11	improve HTTP status code and character encoding support
Jakarta Servlet 6.0	May 31, 2022	6.0	Jakarta EE 10	remove deprecated features and implement requested enhancements
Jakarta Servlet 5.0	October 9, 2020	5.0	Jakarta EE 9	API moved from package <code>javax.servlet</code> to <code>jakarta.servlet</code>
Jakarta Servlet 4.0.3	September 10, 2019	4.0	Jakarta EE 8	Renamed from "Java" trademark
Java Servlet 4.0	September 2017	JSR 369	Java EE 8	HTTP/2
Java Servlet 3.1	May 2013	JSR 340	Java EE 7	Non-blocking I/O, HTTP protocol upgrade mechanism (WebSocket) ^[19]
Java Servlet 3.0	December 2009 ^[20]	JSR 315	Java EE 6	Pluggability, Ease of development, Async Servlet, Security, File Uploading
Java Servlet 2.5	September 2005 ^[21]	JSR 154	Java EE 5	Requires Java SE 5, supports annotation
Java Servlet 2.4	November 2003 ^[22]	JSR 154	J2EE 1.4	web.xml uses XML Schema
Java Servlet 2.3	August 2001 ^[23]	JSR 53	J2EE 1.3	Addition of <code>Filter</code>
Java Servlet 2.2	August 1999 ^[24]	JSR 902 , JSR 903	J2EE 1.2	Becomes part of J2EE, introduced independent web applications in .war files
Java Servlet 2.1	November 1998 ^[25]	2.1a	Unspecified	First official specification, added <code>RequestDispatcher</code> , <code>ServletContext</code>
Java Servlet 2.0	December 1997	—	JDK 1.1	Part of April 1998 Java Servlet Development Kit 2.0 ^[26]
Java Servlet 1.0	December 1996	—		Part of June 1997 Java Servlet Development Kit (JSDK) 1.0 ^[14]

Possible 'X' on the project icon

Should you see an X ...



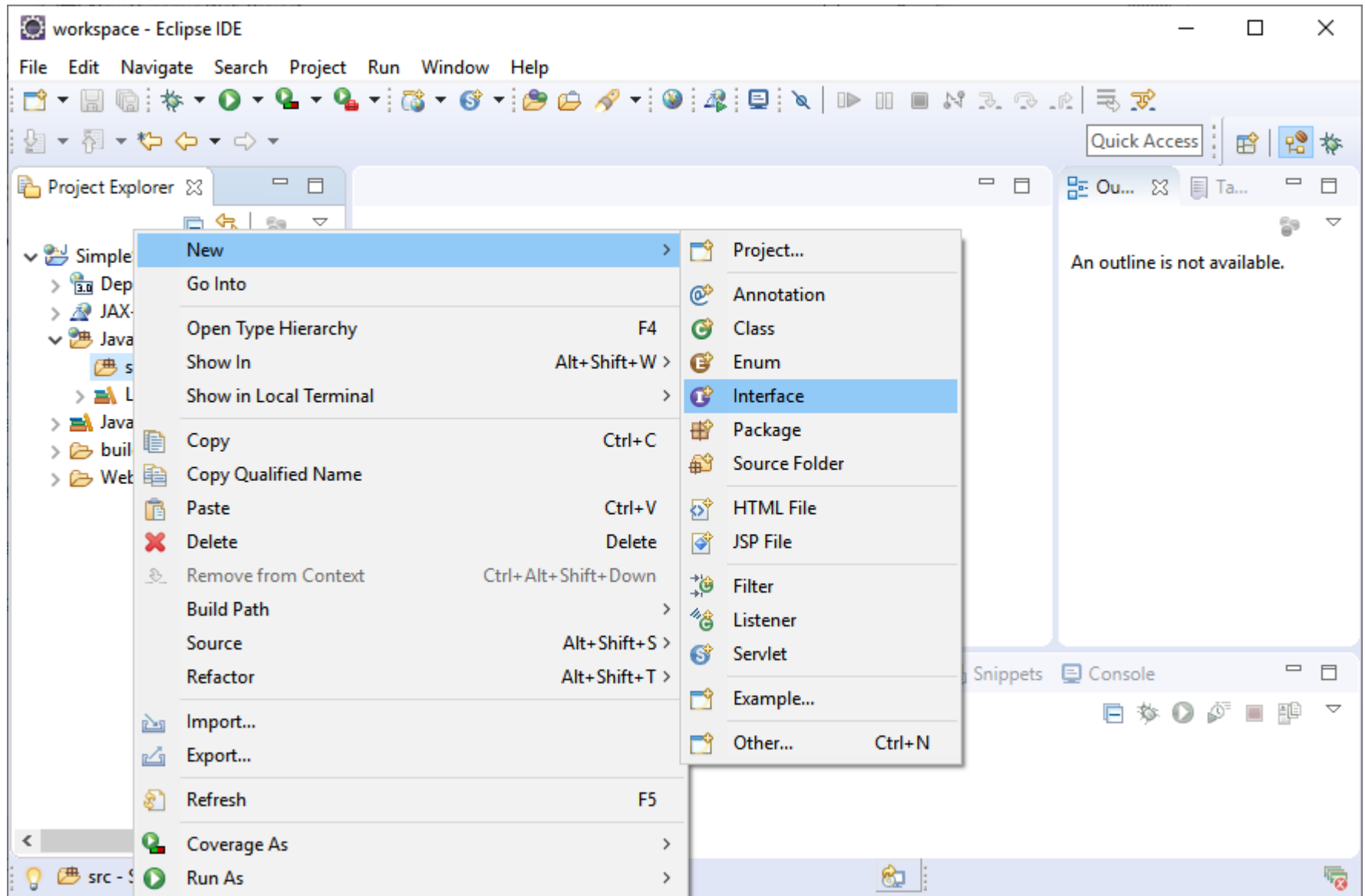
Try to double click the "Error Pages" entry

Resource '/SimpleSumSOAPService/WebContent/WEB-INF/web.xml' does not exist.

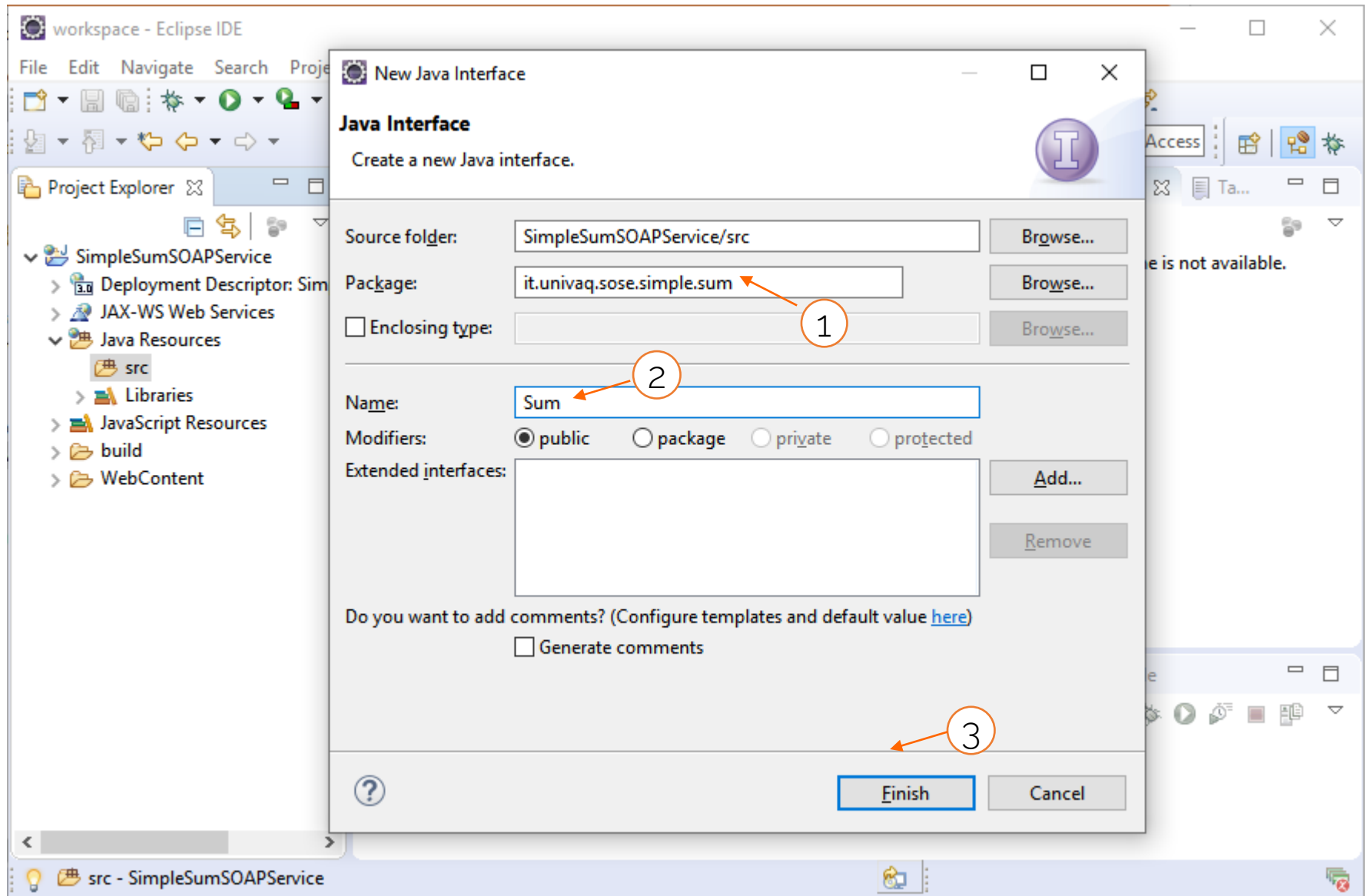
This is due to the fact that I opted for creating a "Dynamic Web Project" but, in previous slide, I did not tell Eclipse to create a basic web.xml file ☺

We will see "how to" in future slides...

Create the Sum Interface



Create the Sum Interface



Create the Sum Interface ... ☹️

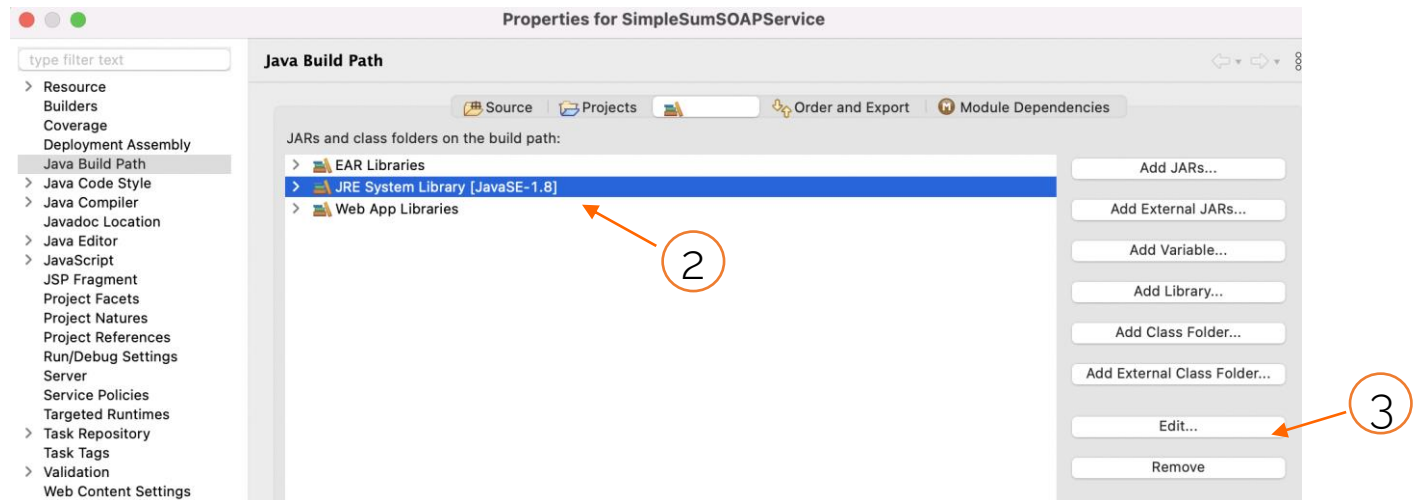
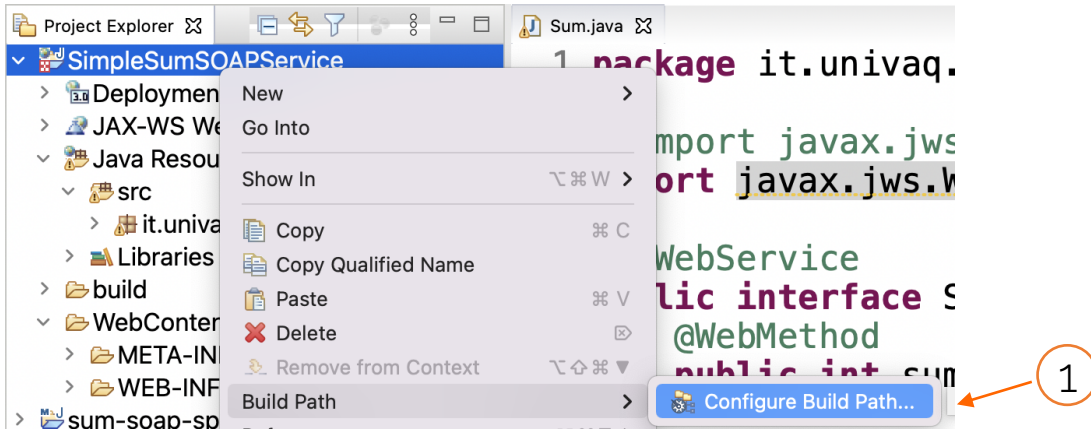
The screenshot shows the Eclipse IDE interface. The title bar reads "workspace - SimpleSumSOAPService/src/it/univaq/sose/simple/sum/Sum.java - Eclipse IDE". The menu bar includes File, Edit, Source, Refactor, Navigate, Search, Project, Run, Window, and Help. The toolbar contains various icons for file operations, running, and debugging. The Project Explorer on the left shows the project structure: SimpleSumSOAPService, Deployment Descriptor: SimpleSumSOAPService, JAX-WS Web Services, Java Resources (src, it.univaq.sose.simple.sum, Sum.java), Libraries, JavaScript Resources, build, and WebContent. The main editor displays the code for Sum.java:

```
1 package it.univaq.sose.simple.sum;
2
3 import javax.xml.ws.WebMethod;
4 import javax.xml.ws.WebService;
5
6 @WebService
7 public interface Sum {
8     @WebMethod
9     public int sum(int a, int b);
10
11 }
```

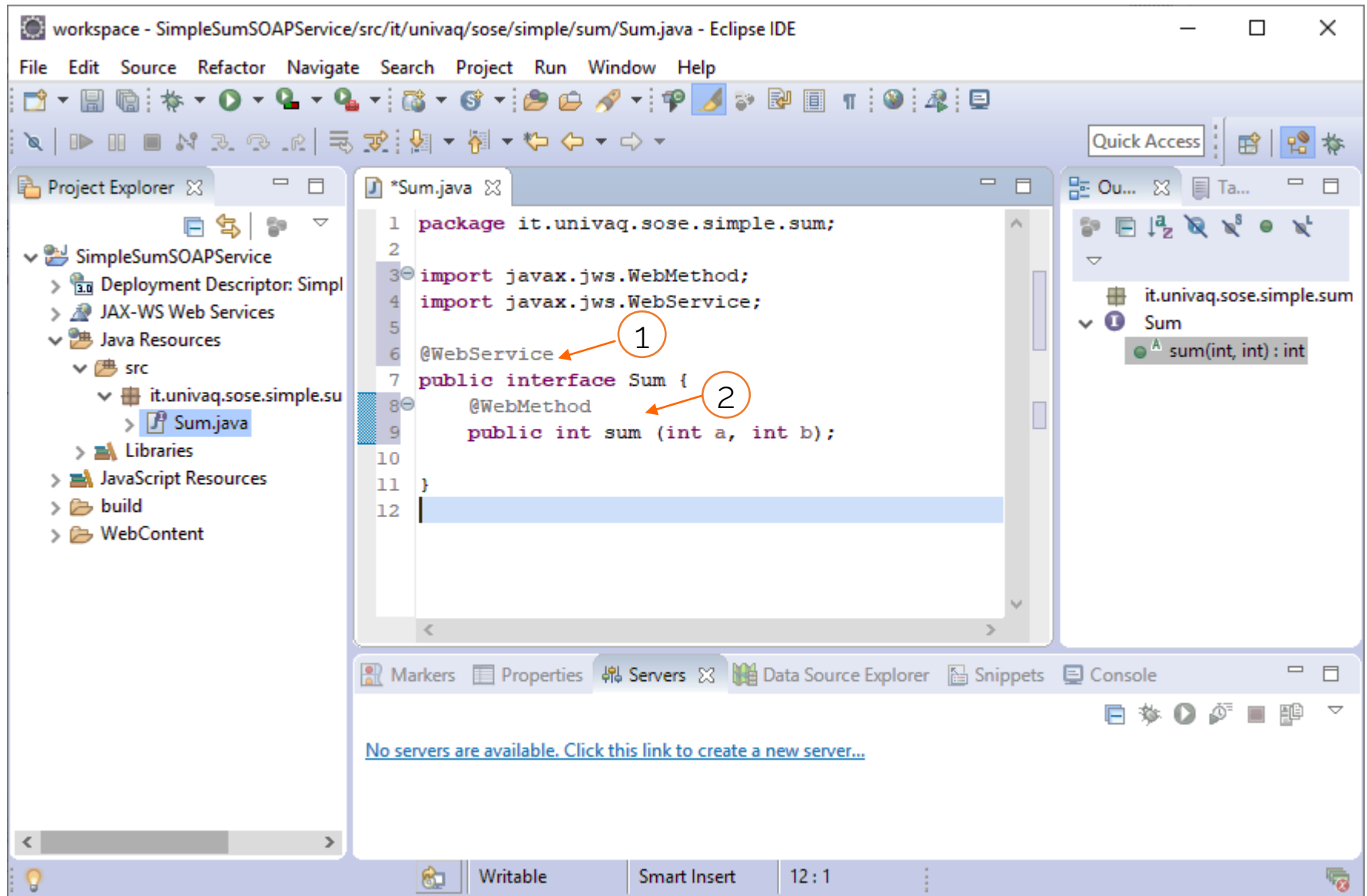
Red squiggly lines under the import statements indicate errors. A tooltip for line 3 says "The import javax.xml.ws cannot be resolved". The Package Explorer on the right shows the package "it.univaq.sose.simple.sum" and the interface "Sum" with a method "sum(int, int) : int". The Console at the bottom shows the message "No servers are available. Click this link to create a new server...".

- Should you have errors like these ones, make sure you chose the right JDK (see previous slides)
- Make also sure that your Java Built Path is properly configured (see next slide)

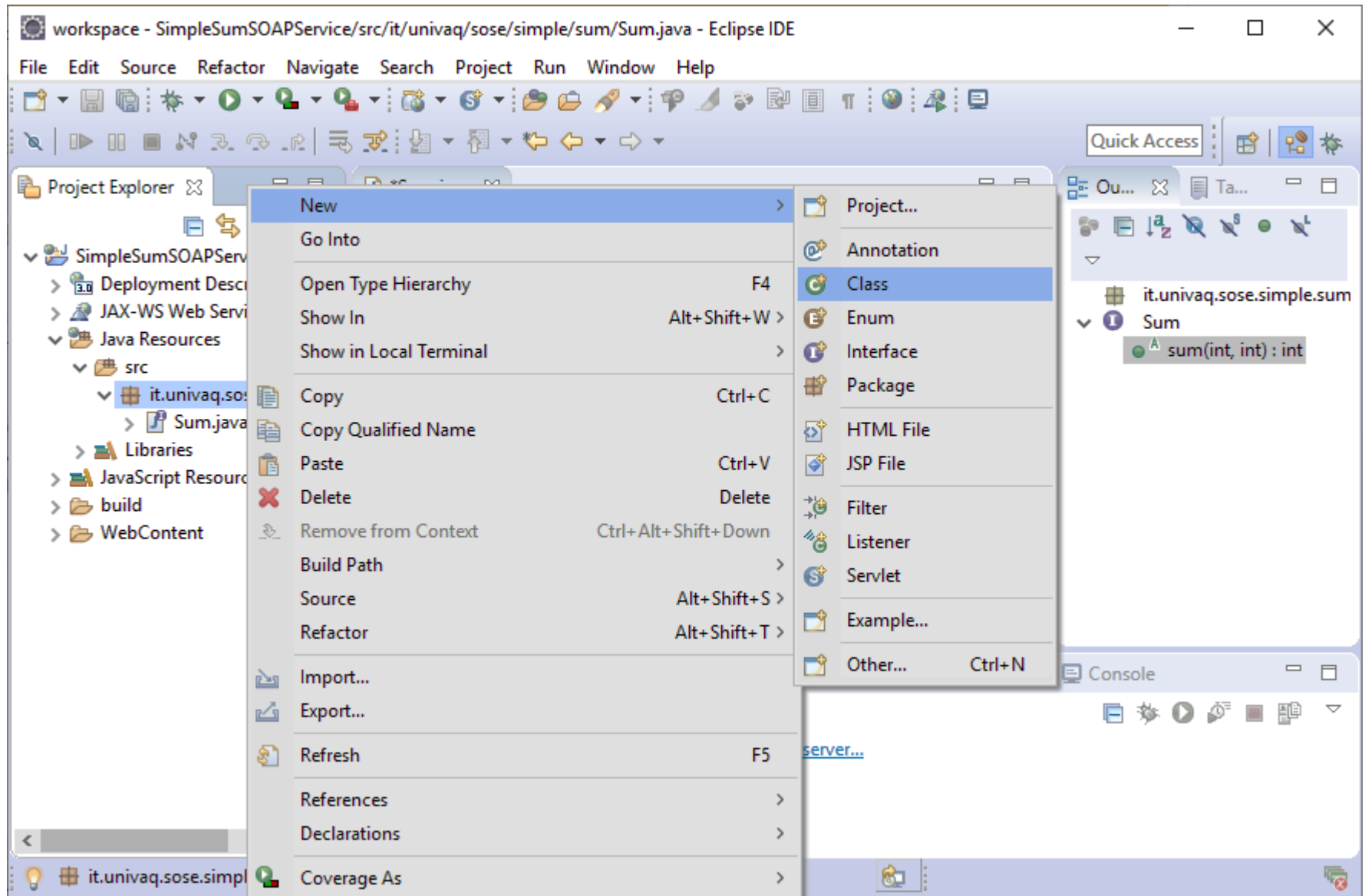
Configure the Java Build Path ☺



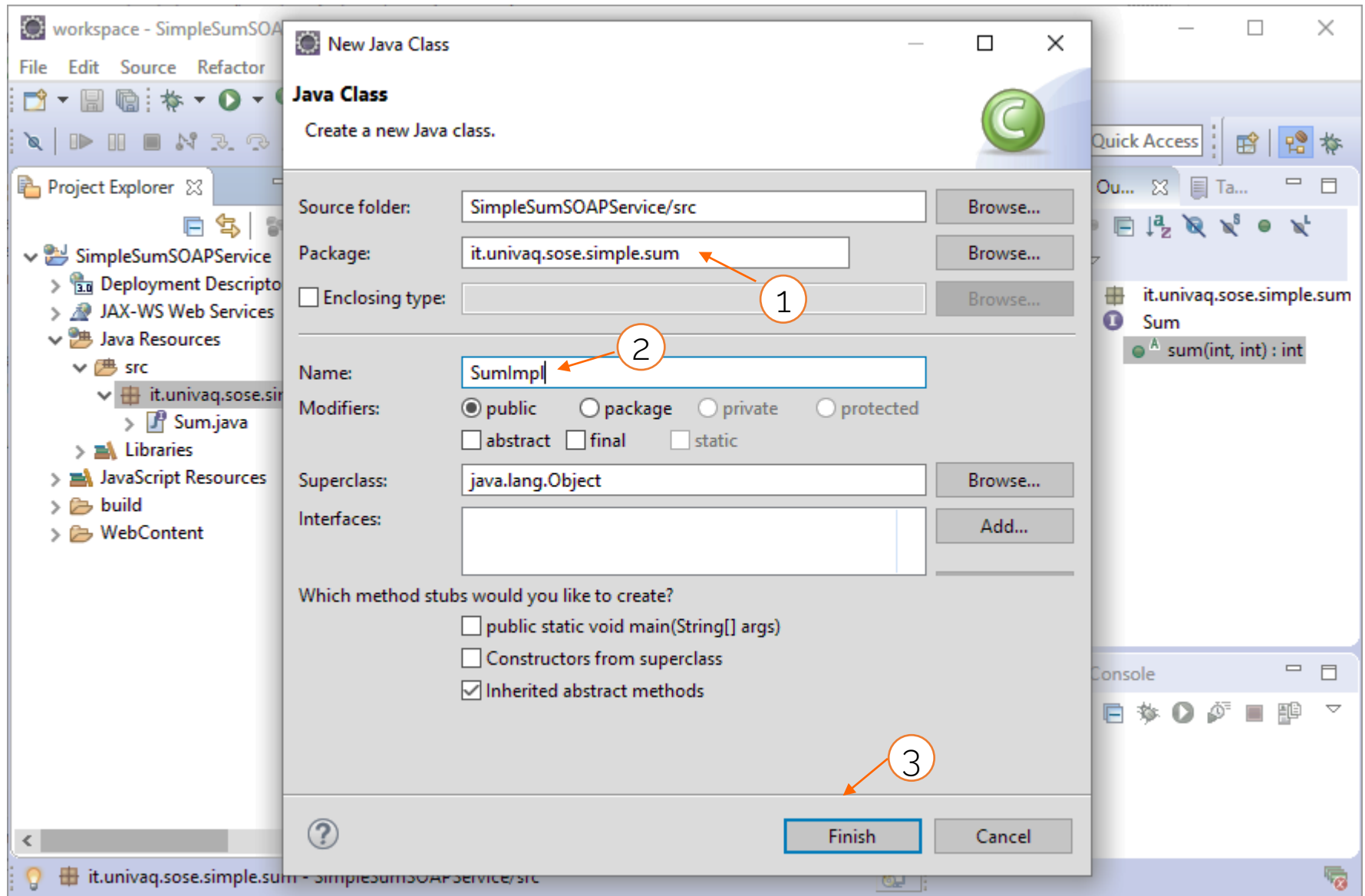
Create the Sum Interface



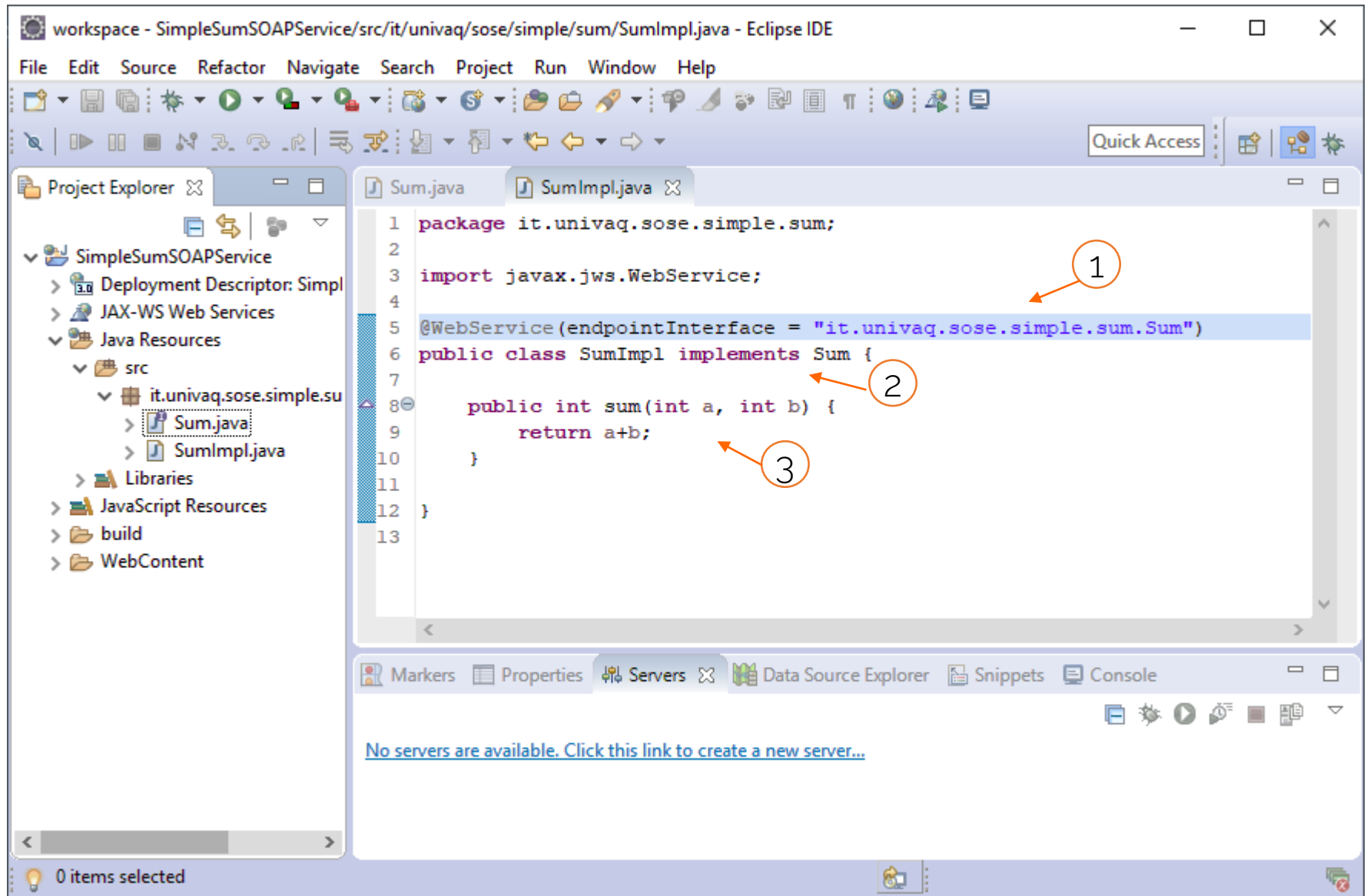
Create the SumImpl Class



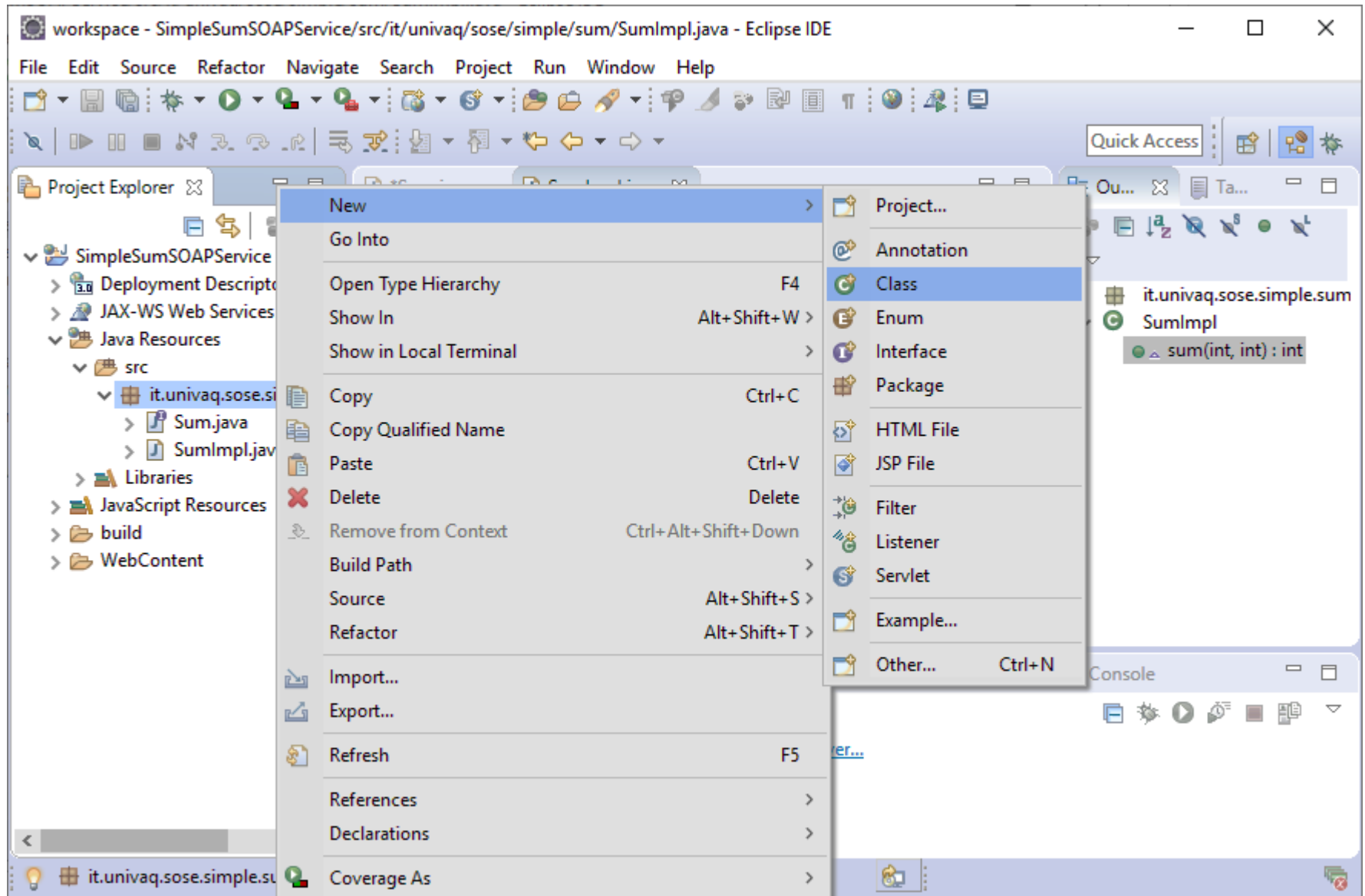
Create the SumImpl Class



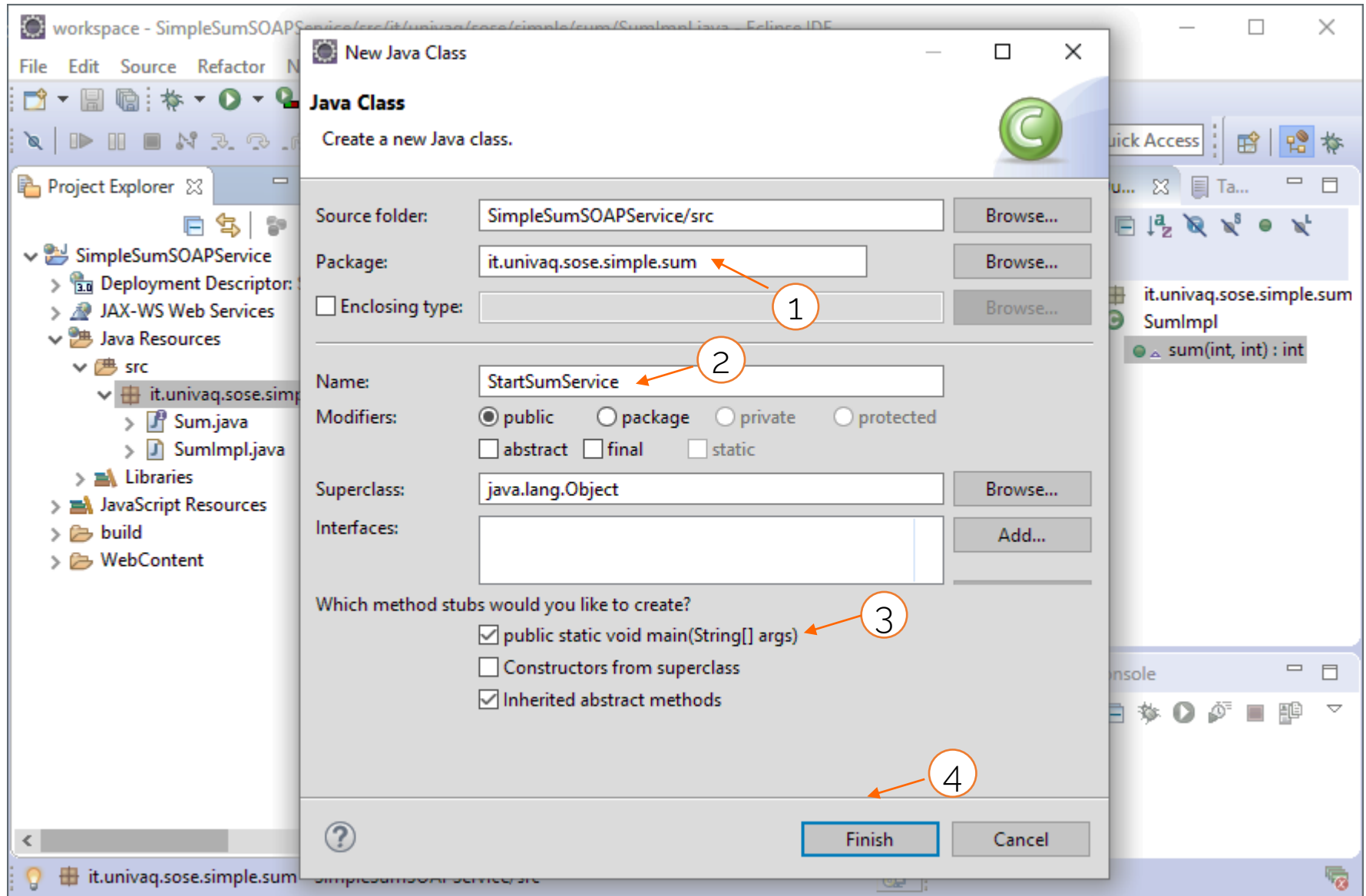
Create the SumImpl Class



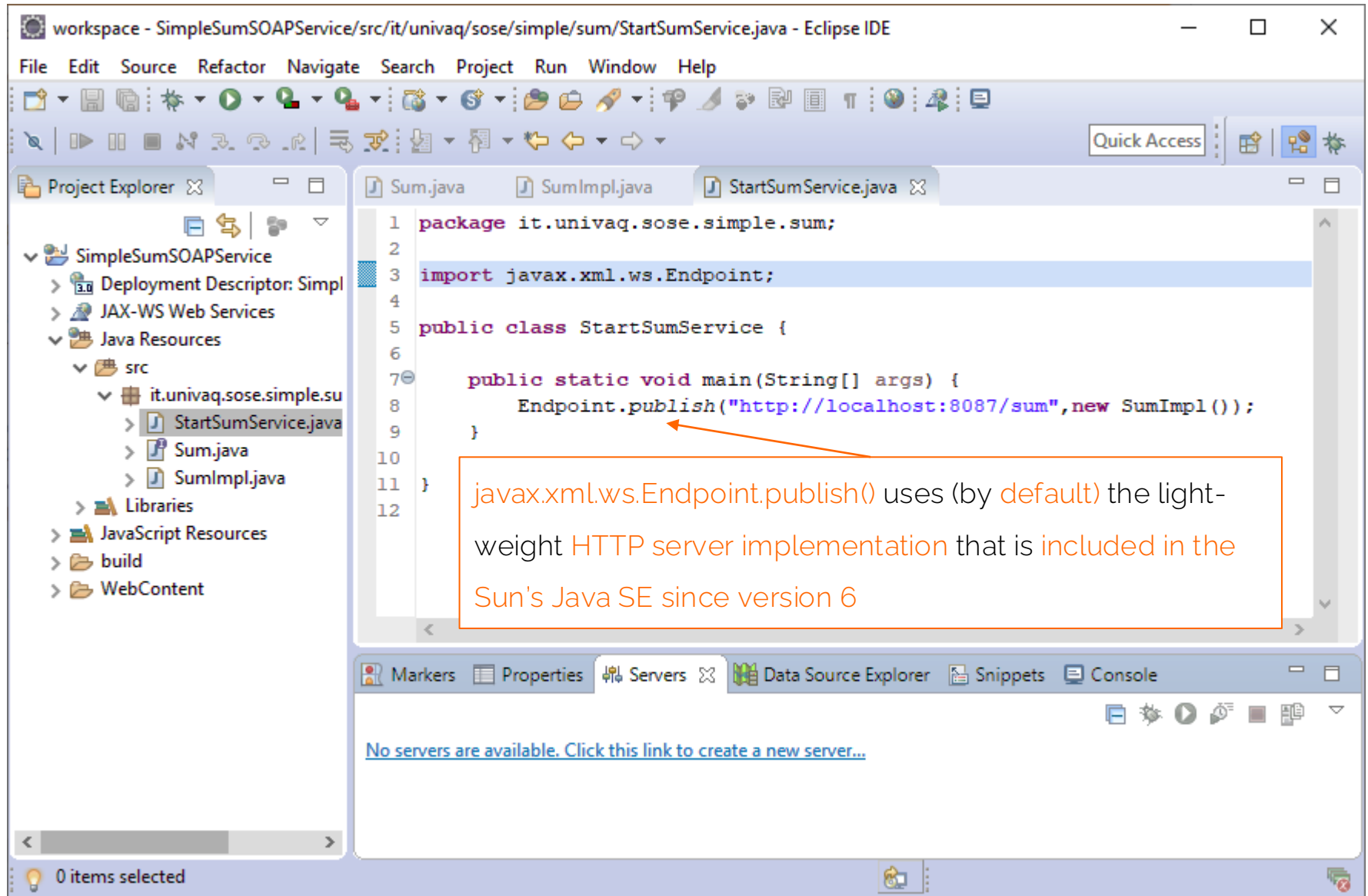
Create the StartSumService Class



Create the StartSumService Class



Create the StartSumService Class



The screenshot shows the Eclipse IDE interface. The title bar reads "workspace - SimpleSumSOAPService/src/it/univaq/sose/simple/sum/StartSumService.java - Eclipse IDE". The menu bar includes File, Edit, Source, Refactor, Navigate, Search, Project, Run, Window, and Help. The toolbar contains various icons for file operations, running, and debugging. The Project Explorer on the left shows the project structure: SimpleSumSOAPService, Deployment Descriptor: SimpleSumSOAPService, JAX-WS Web Services, Java Resources, src, it.univaq.sose.simple.sum, StartSumService.java, Sum.java, SumImpl.java, Libraries, JavaScript Resources, build, and WebContent. The main editor displays the code for StartSumService.java:

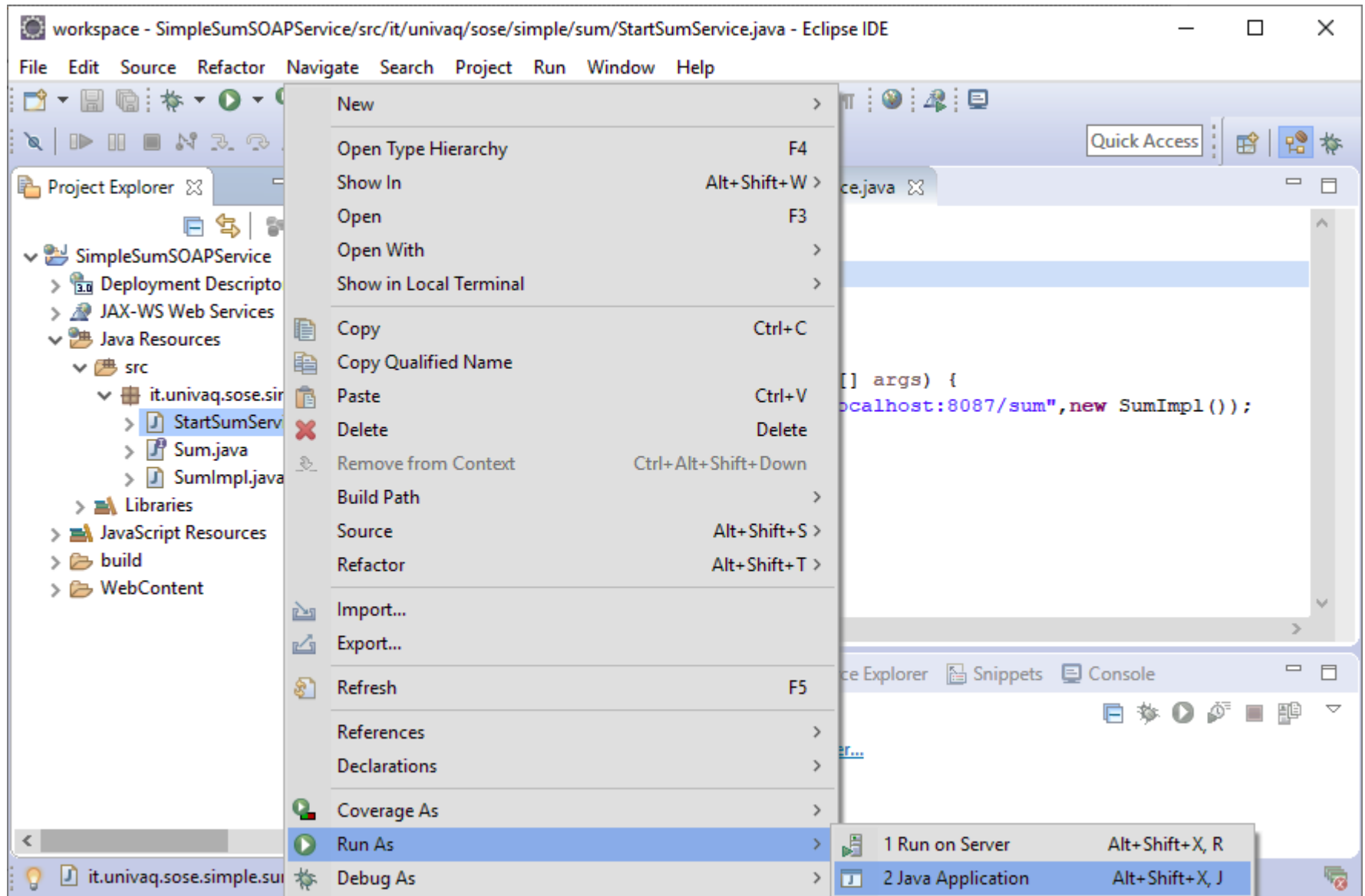
```
1 package it.univaq.sose.simple.sum;
2
3 import javax.xml.ws.Endpoint;
4
5 public class StartSumService {
6
7     public static void main(String[] args) {
8         Endpoint.publish("http://localhost:8087/sum", new SumImpl());
9     }
10 }
11
12
```

An orange arrow points from a text box to the `Endpoint.publish` method call in line 8. The text box contains the following text:

`javax.xml.ws.Endpoint.publish()` uses (by default) the lightweight HTTP server implementation that is included in the Sun's Java SE since version 6

The bottom of the IDE shows the Markers, Properties, Servers, Data Source Explorer, Snippets, and Console tabs. The Console tab is active and displays the message: "No servers are available. Click this link to create a new server...". The status bar at the bottom indicates "0 items selected".

Run the StartSumService Class



The Service is up (WSDL)

1

localhost:8087/sum?wsdl

This XML file does not appear to have any style information associated with it. The document tree is shown below.

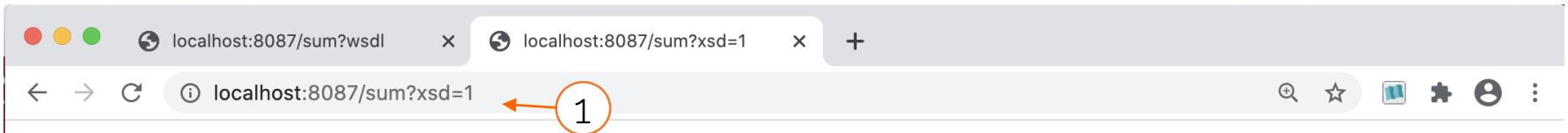
```
<!-- Published by JAX-WS RI (http://jax-ws.java.net). RI's version is JAX-WS RI 2.2.9-b130926.103
<!-- Generated by JAX-WS RI (http://jax-ws.java.net). RI's version is JAX-WS RI 2.2.9-b130926.103
▼<definitions xmlns:wsu="http://docs.oasis-open.org/wss/2004/01/oasis-200401-wss-wssecurity-
utility-1.0.xsd" xmlns:wsp="http://www.w3.org/ns/ws-policy"
xmlns:wsp1_2="http://schemas.xmlsoap.org/ws/2004/09/policy"
xmlns:wsam="http://www.w3.org/2007/05/addressing/metadata"
xmlns:soap="http://schemas.xmlsoap.org/wsdl/soap/" xmlns:tns="http://sum.simple.sose.univaq.it/"
xmlns:xsd="http://www.w3.org/2001/XMLSchema" xmlns="http://schemas.xmlsoap.org/wsdl/"
targetNamespace="http://sum.simple.sose.univaq.it/" name="SumImplService">
▼<types>
▼<xsd:schema>
<xsd:import namespace="http://sum.simple.sose.univaq.it/"
schemaLocation="http://localhost:8087/sum?xsd=1"/>
</xsd:schema>
</types>
▼<message name="sum">
<part name="parameters" element="tns:sum"/>
</message>
▼<message name="sumResponse">
<part name="parameters" element="tns:sumResponse"/>
</message>
▼<portType name="Sum">
▼<operation name="sum">
<input wsam:Action="http://sum.simple.sose.univaq.it/Sum/sumRequest" message="tns:sum"/>
<output wsam:Action="http://sum.simple.sose.univaq.it/Sum/sumResponse"
message="tns:sumResponse"/>
</operation>
</portType>
```

The Service is up (WSDL)



```
<?xml version='1.0' encoding='UTF-8'?>
<binding name="SumImplPortBinding" type="tns:Sum">
  <soap:binding transport="http://schemas.xmlsoap.org/soap/http" style="document"/>
  <operation name="sum">
    <soap:operation soapAction=""/>
    <input>
      <soap:body use="literal"/>
    </input>
    <output>
      <soap:body use="literal"/>
    </output>
  </operation>
</binding>
<service name="SumImplService">
  <port name="SumImplPort" binding="tns:SumImplPortBinding">
    <soap:address location="http://localhost:8087/sum"/>
  </port>
</service>
</definitions>
```

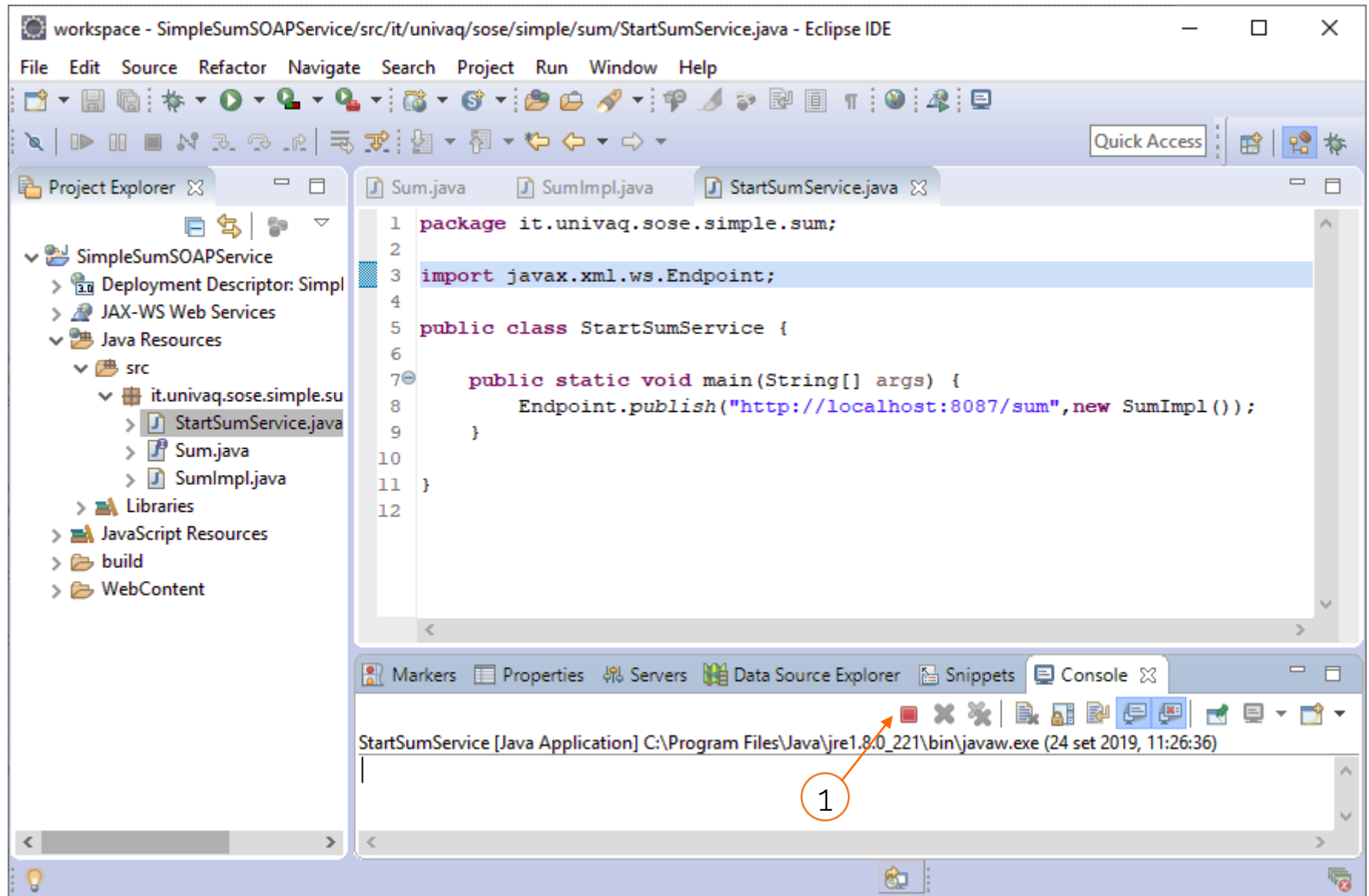
The Service is up (XSD)



This XML file does not appear to have any style information associated with it. The document tree is shown below.

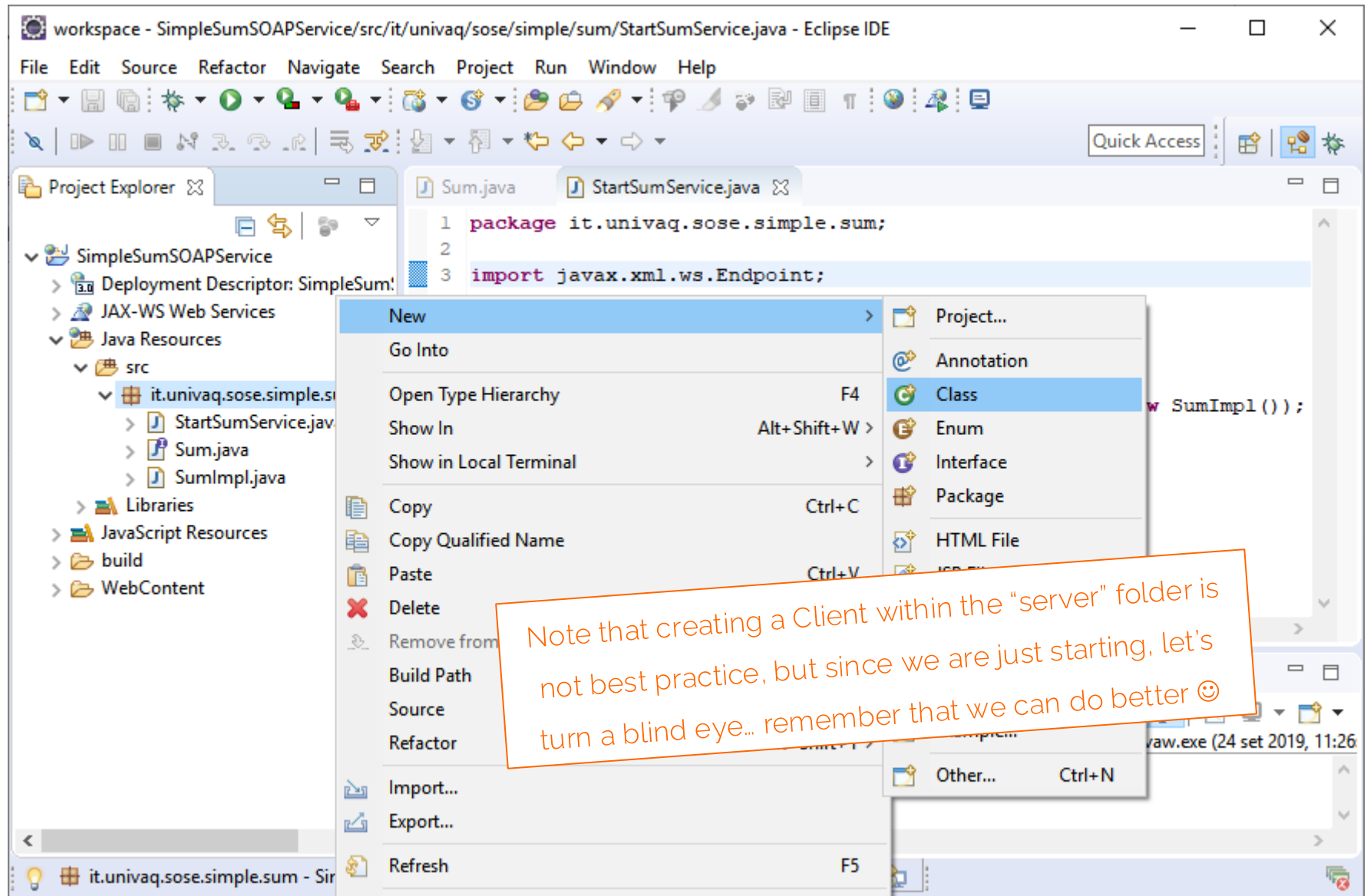
```
<!-- Published by JAX-WS RI (http://jax-ws.java.net). RI's version is JAX-WS
<xs:schema xmlns:tns="http://sum.simple.rose.univaq.it/"
xmlns:xs="http://www.w3.org/2001/XMLSchema" version="1.0"
targetNamespace="http://sum.simple.rose.univaq.it/">
  <xs:element name="sum" type="tns:sum" />
  <xs:element name="sumResponse" type="tns:sumResponse" />
  <xs:complexType name="sum">
    <xs:sequence>
      <xs:element name="arg0" type="xs:int" />
      <xs:element name="arg1" type="xs:int" />
    </xs:sequence>
  </xs:complexType>
  <xs:complexType name="sumResponse">
    <xs:sequence>
      <xs:element name="return" type="xs:int" />
    </xs:sequence>
  </xs:complexType>
</xs:schema>
```

Stop the Service

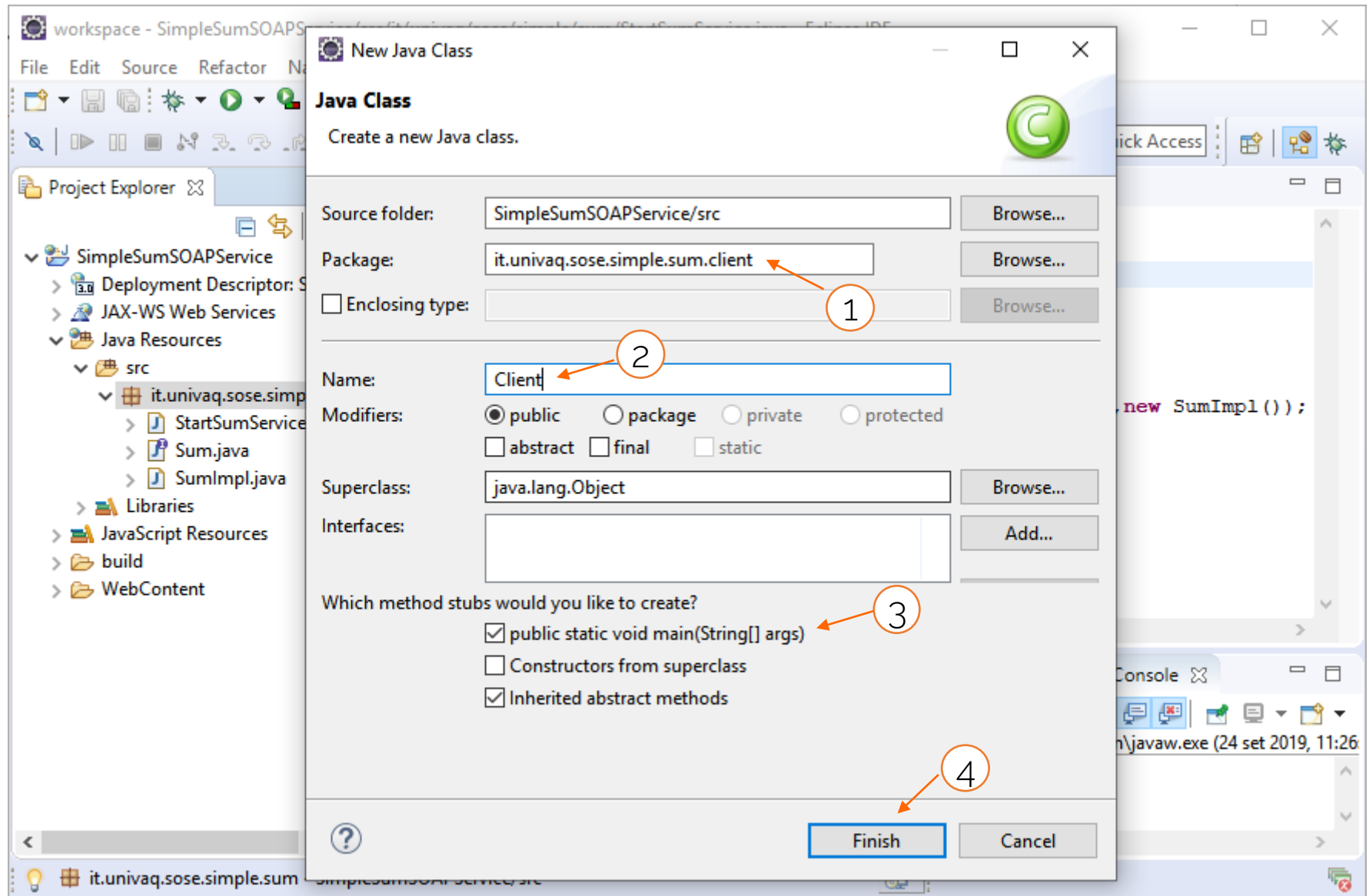


How can we invoke our SOAP service?

Create the Client Class



Create the Client Class



Create the Client Class

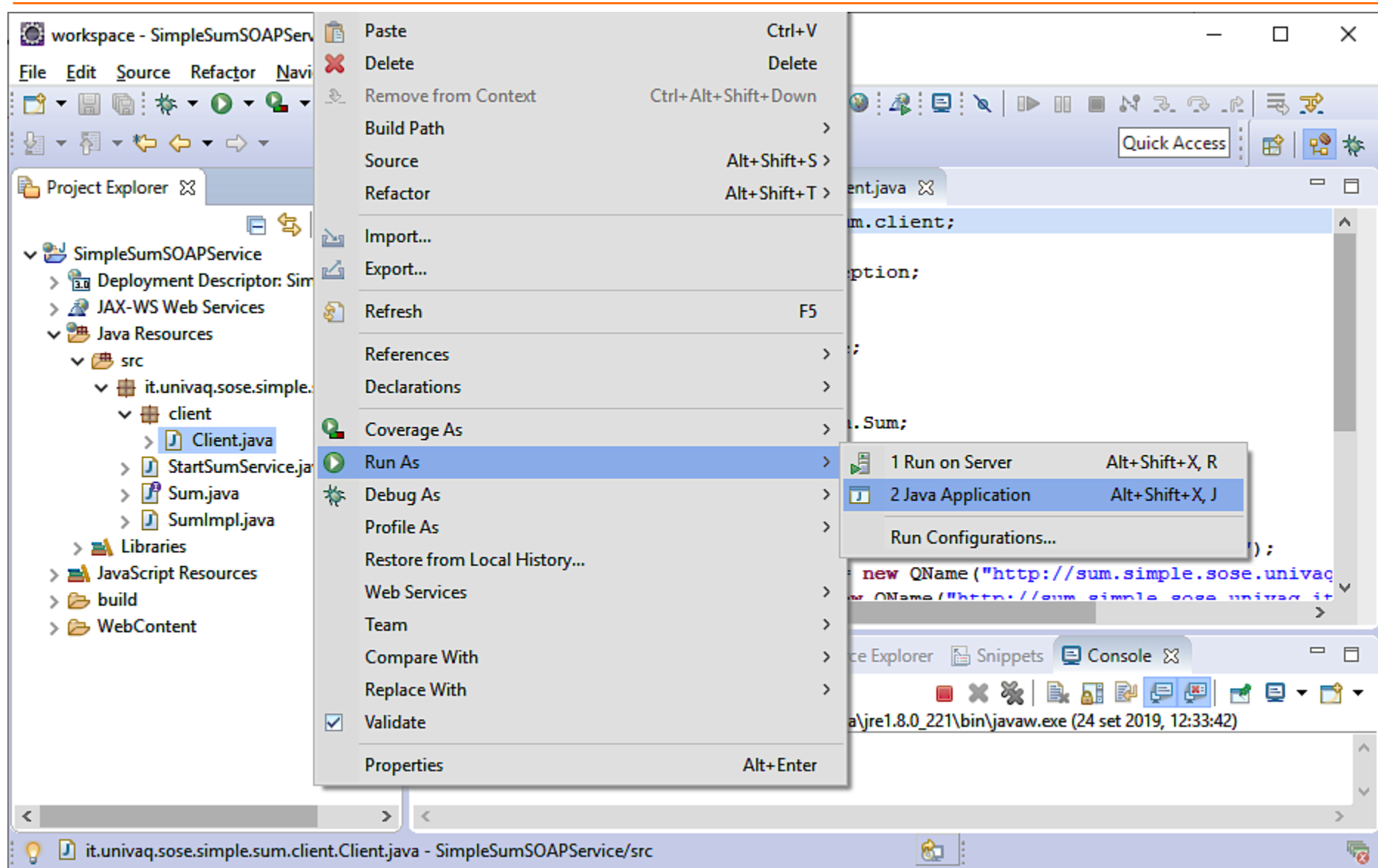
The screenshot shows an IDE with the following components:

- WSDL Definition (Top):** A snippet of WSDL code defining the `SumImplService`. It includes namespaces for `wsu`, `wsp`, `wsam`, `soap`, `tns`, and `xsd`. The `targetNamespace` is `http://sum.simple.sose.univaq.it/` and the service name is `SumImplService`.
- Client.java Code (Middle):** The code defines a `Client` class with a `main` method. It imports `java.net.MalformedURLException`, `java.net.URL`, `javax.xml.namespace.QName`, `javax.xml.ws.Service`, and `it.univaq.sose.simple.sum.Sum`. The `main` method creates a `URL` for the WSDL, creates a `QName` for the service and port, creates a `Service` object, gets the `Sum` port, and calls `sum(5, 5)`. It also prints the result and catches `MalformedURLException`.
- WSDL Port Definition (Bottom):** A snippet of WSDL code defining the `SumImplPort` binding. It shows the `port` name, the `binding` name, and the `soap:address` location.

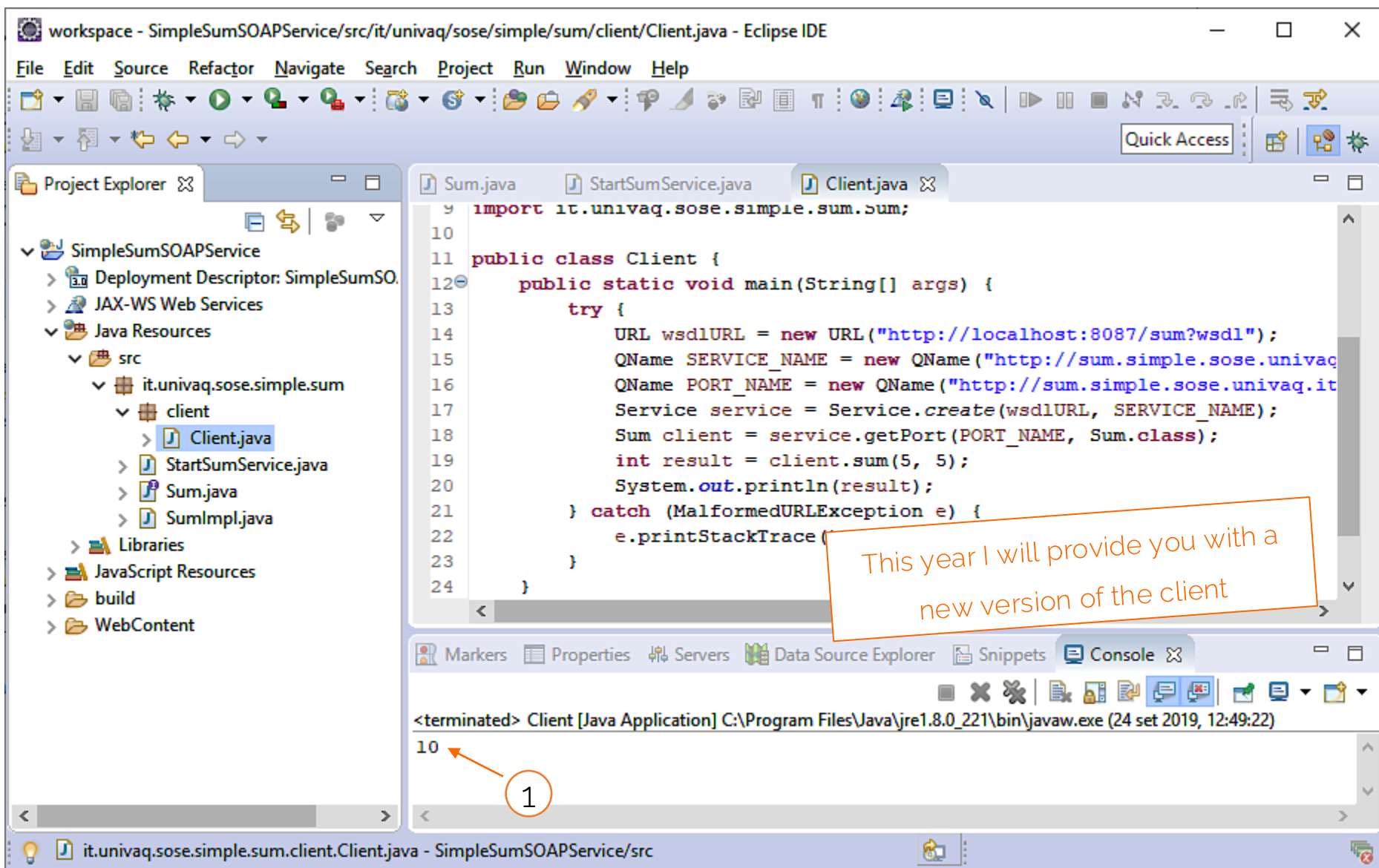
Annotations in the image:

- A box around the WSDL definition at the top.
- A box around the `Service` creation and port retrieval code in `Client.java`.
- A box around the `SumImplPort` binding definition at the bottom.
- Arrows pointing from the `SumImplService` name in the WSDL to the `SERVICE_NAME` variable in the code.
- Arrows pointing from the `SumImplPort` name in the WSDL to the `PORT_NAME` variable in the code.
- A box with the text "SOAP-based message-oriented invocation of a remote operation" pointing to the `Service` creation and port retrieval code.

Run the Client Class



The Service is Invoked



What happened behind the scene?

- SOAP
- Handlers

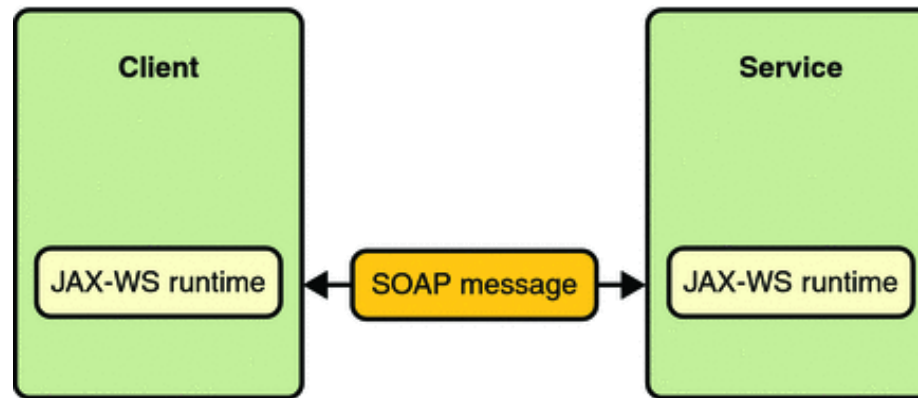
What is SOAP?

It stands for Simple Object Access Protocol

It is a W3C recommendation

It is an application communication protocol based on XML

It is a format for sending and receiving messages



Advantages and disadvantages

Advantages

- Human readable XML
- Easy to debug
- SOAP runs over HTTP
- Firewall not affected
- It is platform and language independent

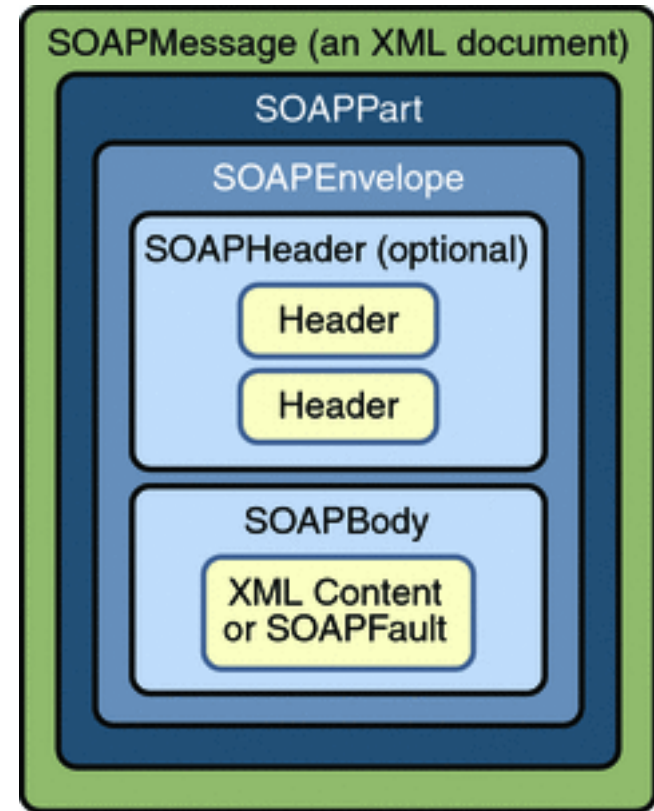
Disadvantages

- XML produces a lot of overhead for small messages
- SOAP has different levels of support, depending upon the programming language supported

SOAP structure

A SOAP message is made of

- An **Envelope element** (mandatory) that identifies the XML document as a SOAP message
- A **Header element** (optional) that contains header information
- A **Body element** (mandatory) that contains request and response information
- A **Fault element** (optional) containing errors and status information. It is part of the **Body element**



SOAP skeleton message

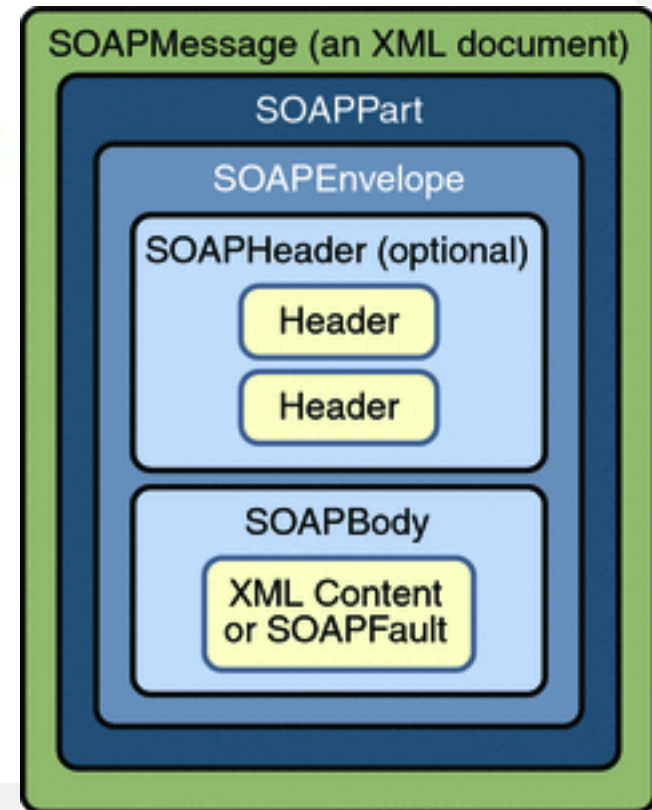
```
<?xml version="1.0"?>

<soap:Envelope
xmlns:soap="http://www.w3.org/2003/05/soap-envelope/"
soap:encodingStyle="http://www.w3.org/2003/05/soap-encoding">

  <soap:Header>
    ...
  </soap:Header>

  <soap:Body>
    ...
    <soap:Fault>
      ...
    </soap:Fault>
  </soap:Body>

</soap:Envelope>
```



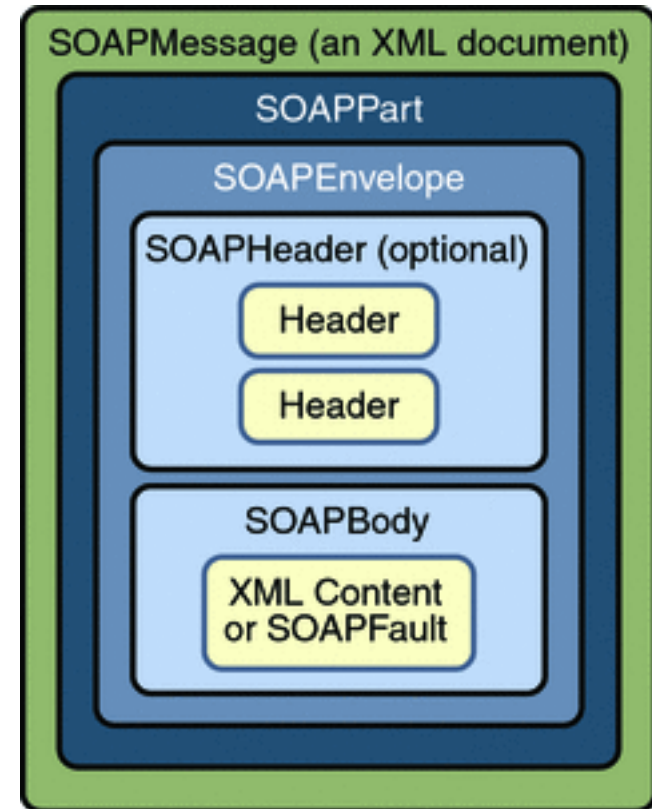
Envelope element

The SOAP envelope indicates the start and the end of the message so that the receiver knows when an entire message has been received

Therefore, the SOAP envelope is basically a packaging mechanism

Some characteristics

- It is a mandatory part of SOAP message
- It is the root of the SOAP message
- It contains the declaration of SOAP namespace
- It must contain exactly one Body element



Header element

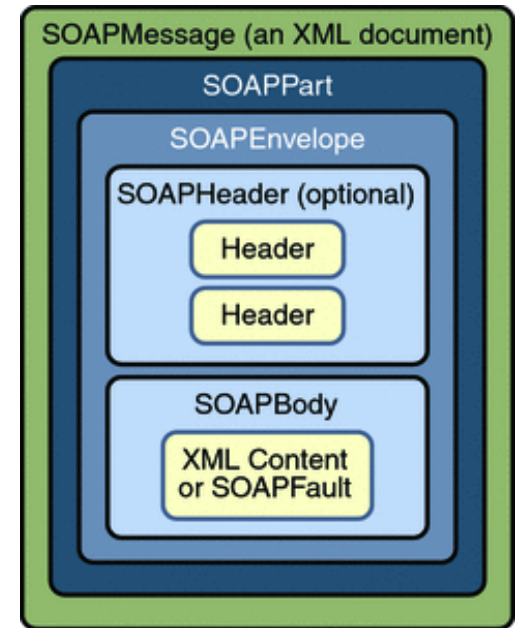
The **optional Header element** offers a flexible framework for specifying **additional application-level requirements**

For example, the Header element can be used to specify a **digital signature for password-protected services**

Likewise, it can be used to specify **an account number for pay-per-use SOAP services**

The Header element **must be the first child** of Envelope

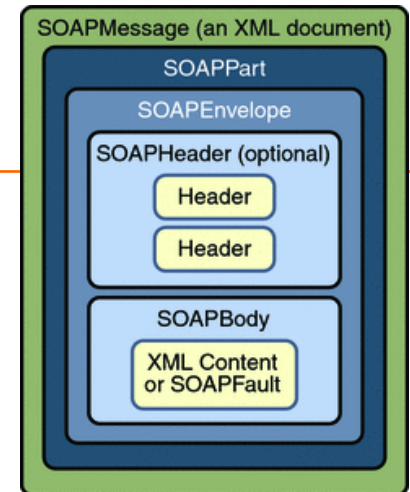
All the Header's sons will be considered as a Header entries



Header element example

e.g., how to specify security information, such as the Username and Password

```
<?xml version="1.0" ?>
<S:Envelope xmlns:S="http://schemas.xmlsoap.org/soap/envelope/" xmlns:SOAP-ENV="http://schemas.xmlsoap.org/soap/envelope/">
  <SOAP-ENV:Header>
    <wsse:Security xmlns:wsse="http://docs.oasis-open.org/wss/2004/01/oasis-200401-wss-wssecurity-secext-1.0.xsd">
      <wsse:UsernameToken xmlns:wsu="http://docs.oasis-open.org/wss/2004/01/oasis-200401-wss-wssecurity-utility-1.0.xsd">
        <wsse:Username>
          TestUser
        </wsse:Username>
        <wsse:Password Type="http://docs.oasis-open.org/wss/2004/01/oasis-200401-wss-username-token-profile-1.0#PasswordText">
          TestPassword
        </wsse:Password>
      </wsse:UsernameToken>
    </wsse:Security>
  </SOAP-ENV:Header>
  <S:Body>
    <ns2:getGreeting xmlns:ns2="http://firstserver.com/">
      <arg0 xmlns="">Riccardo</arg0>
    </ns2:getGreeting>
  </S:Body>
</S:Envelope>
```



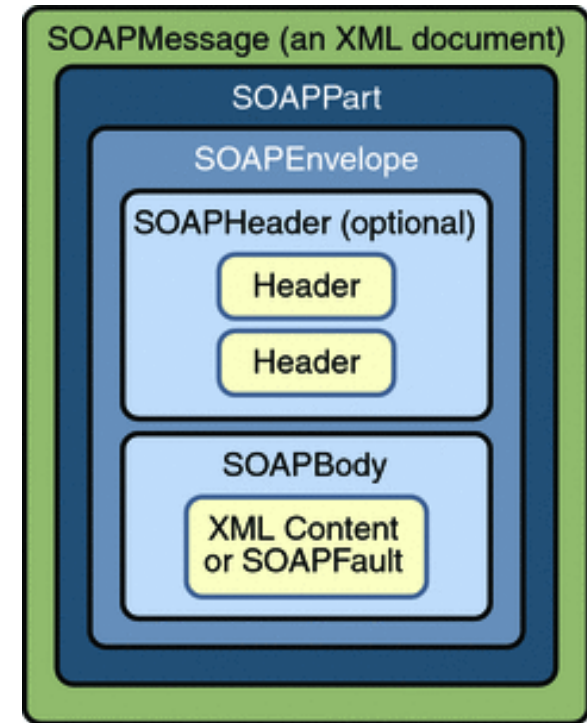
Body element

The **SOAP body** is a **mandatory** element that contains the **application-defined XML data** being exchanged in the SOAP message

The body must **be contained within the envelope and must follow any headers** that might be defined for the message

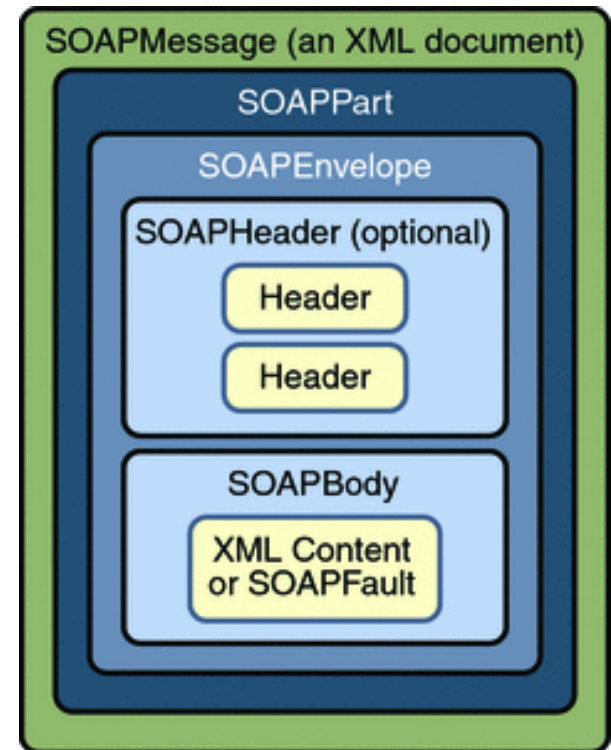
The **semantics** for the body are defined in the associated **SOAP schema**

The body **contains mandatory information** intended for the **receiver of the message**



Body element example

```
10 <?xml version="1.0" ?>
11 <S:Envelope xmlns:S="http://schemas.xmlsoap.org/soap/envelope/" xmlns:SOAP-ENV="http://schemas.xmlsoap.org/soap/envelope/">
12   <SOAP-ENV:Header>
13     <wsse:Security xmlns:wsse="http://docs.oasis-open.org/wss/2004/01/oasis-200401-wss-wssecurity-secext-1.0.xsd">
14       <wsse:UsernameToken xmlns:wsu="http://docs.oasis-open.org/wss/2004/01/oasis-200401-wss-wssecurity-utility-1.0.xsd">
15         <wsse:Username>
16           TestUser
17         </wsse:Username>
18         <wsse:Password Type="http://docs.oasis-open.org/wss/2004/01/oasis-200401-wss-username-token-profile-1.0#PasswordText">
19           TestPassword
20         </wsse:Password>
21       </wsse:UsernameToken>
22     </wsse:Security>
23   </SOAP-ENV:Header>
24   <S:Body>
25     <ns2:getGreeting xmlns:ns2="http://firstserver.com/">
26       <arg0 xmlns="">Riccardo</arg0>
27     </ns2:getGreeting>
28   </S:Body>
29 </S:Envelope>
```

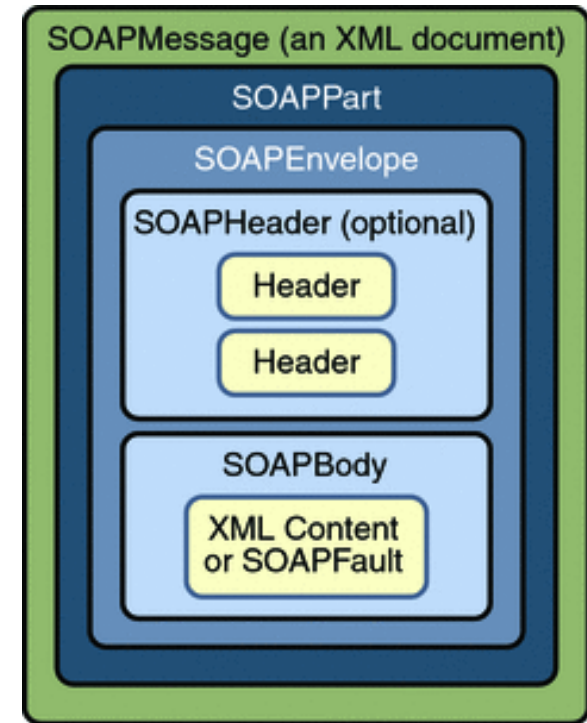


Fault element

The Fault section **occurs only in the response message** if the receiver or the intermediary were not able to work his own part of the message

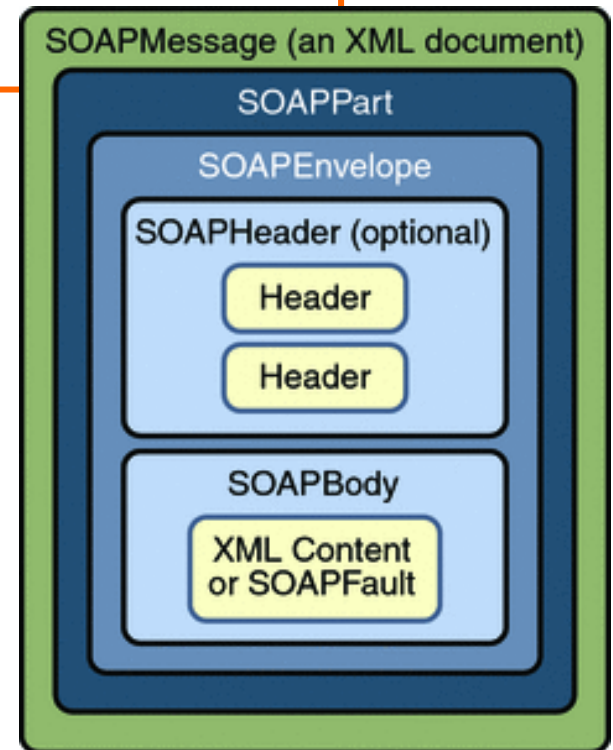
If present, the fault section must be **one of the children of the Body** and can appear only once

The section provide information about the **errors that occurred during the processing of the message**



Fault element example

```
1 <soap:Envelope xmlns:soap="http://schemas.xmlsoap.org/soap/envelope/">
2   <soap:Body>
3     <soap:Fault>
4       <faultcode>soap:Server</faultcode>
5       <faultstring>integer overflow</faultstring>
6       <detail>
7         <ns1:SumException xmlns:ns1="http://sum.odws.univaq.it/">
8       </detail>
9     </soap:Fault>
10  </soap:Body>
11 </soap:Envelope>
```



Handlers

- JAX-WS is equipped with a plugin-based framework, called **Handler**, for processing I/O messages
- The framework permits to **extend the capabilities of the JAX-WS runtime**
- The handler permits to **act on both the client side and the server server**
- Handlers are organized into an ordered list called **handler chain**
- A **handler chain is invoked upon receiving or sending a message**
- Thus, **handlers are message interceptors** that can be easily plugged in to the JAX-WS runtime to do additional processing of the **inbound** and **outbound** messages:
 - Handlers are invoked with a **context message** that provides methods for accessing and modifying **inbound** and **outbound messages**

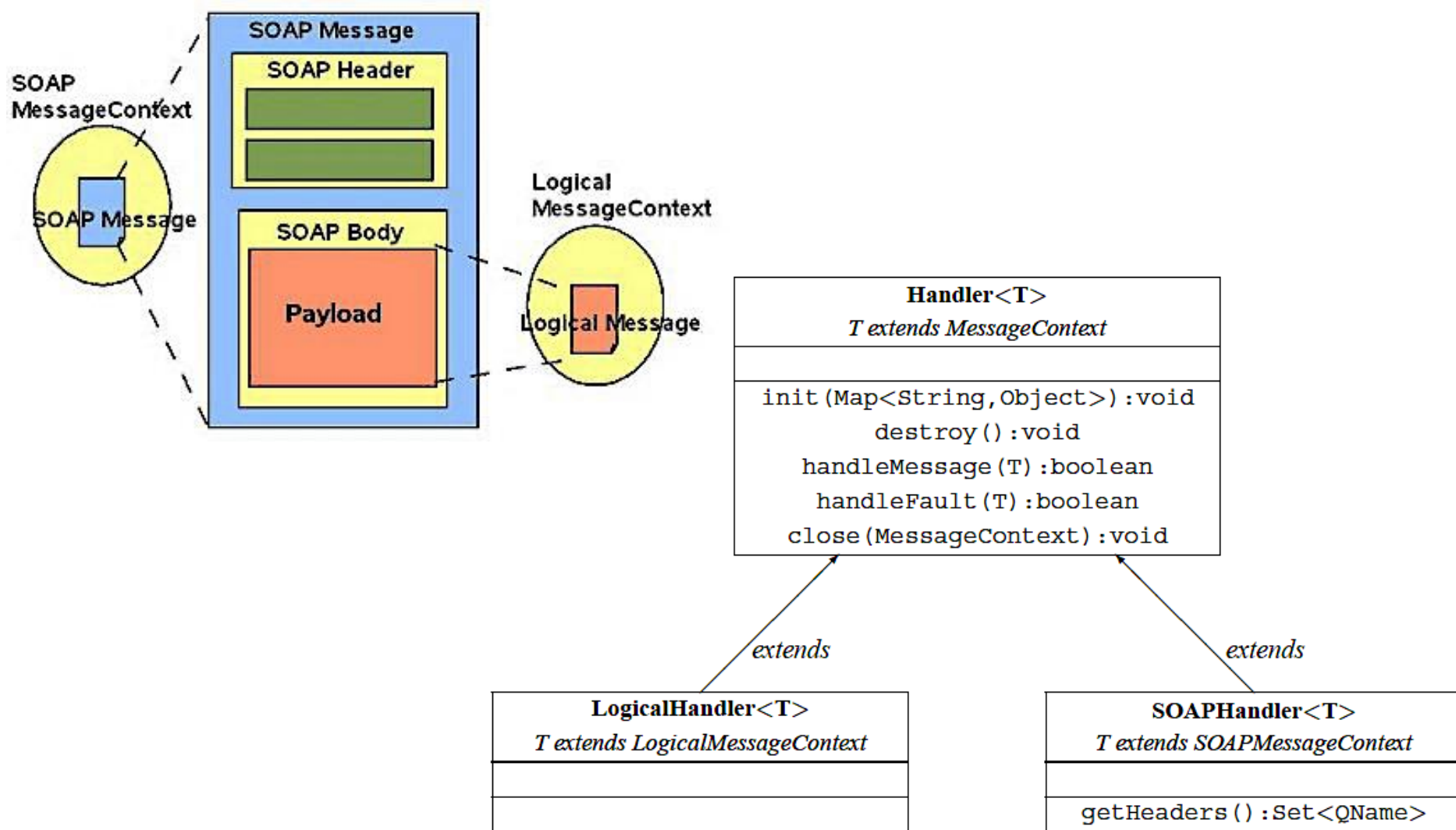
Handlers

JAX-WS defines two types of handlers

- **Logical handlers** are protocol-agnostic and cannot change any protocol-specific parts (like headers) of a message
 - extends `javax.xml.ws.handler.LogicalHandler` and provide **access** to **message context** and **message payload**
- **SOAP handlers** are specific to a protocol and may access or change the protocol specific aspects of a message
 - Implements any interfaces deriving from `javax.xml.ws.handler.Handler` (except for `javax.xml.ws.handler.LogicalHandler`)

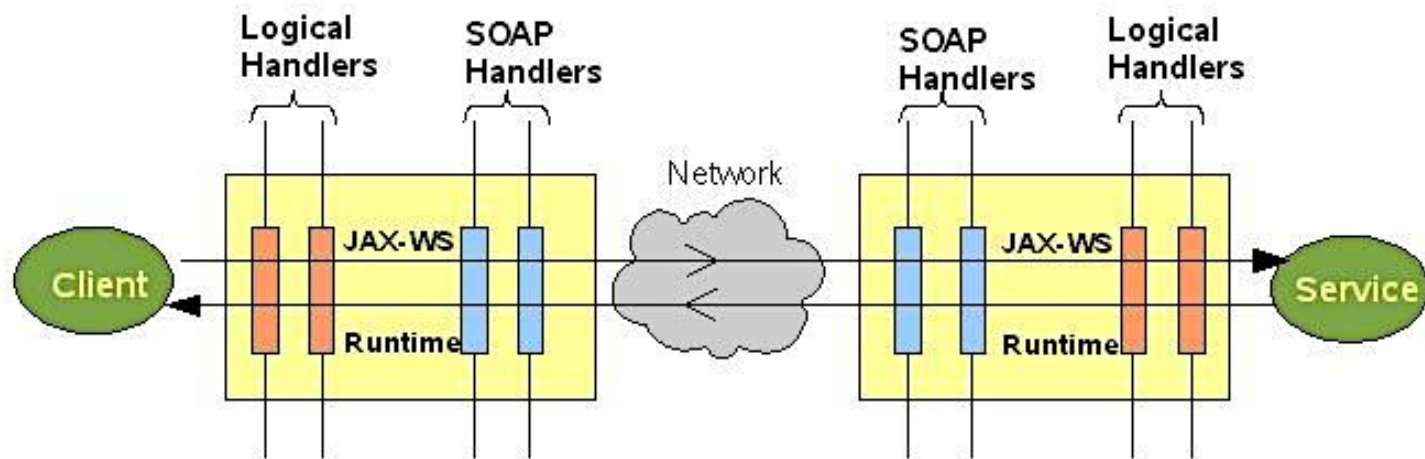
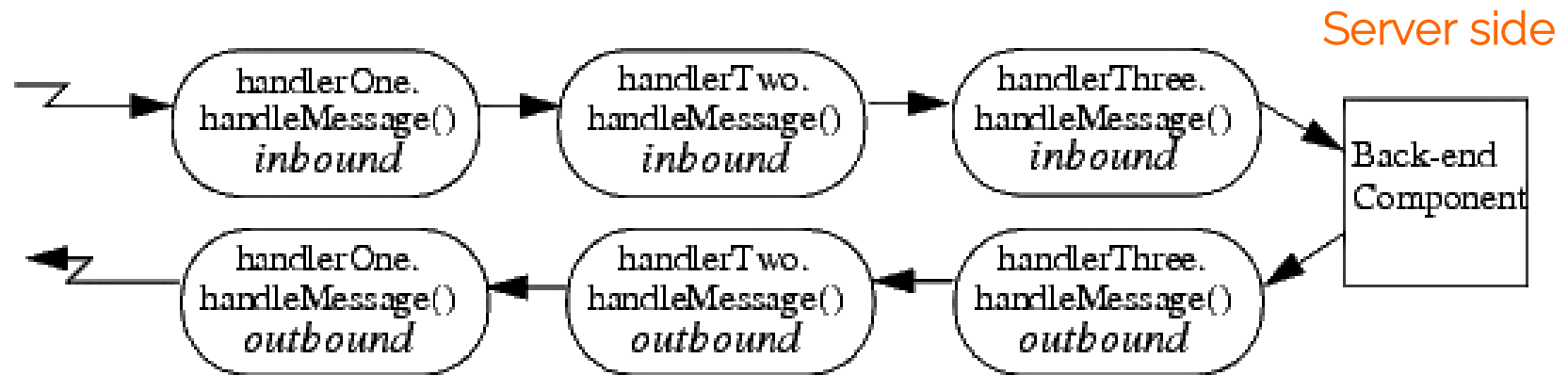
https://jax-ws.java.net/articles/handlers_introduction.html

Handlers

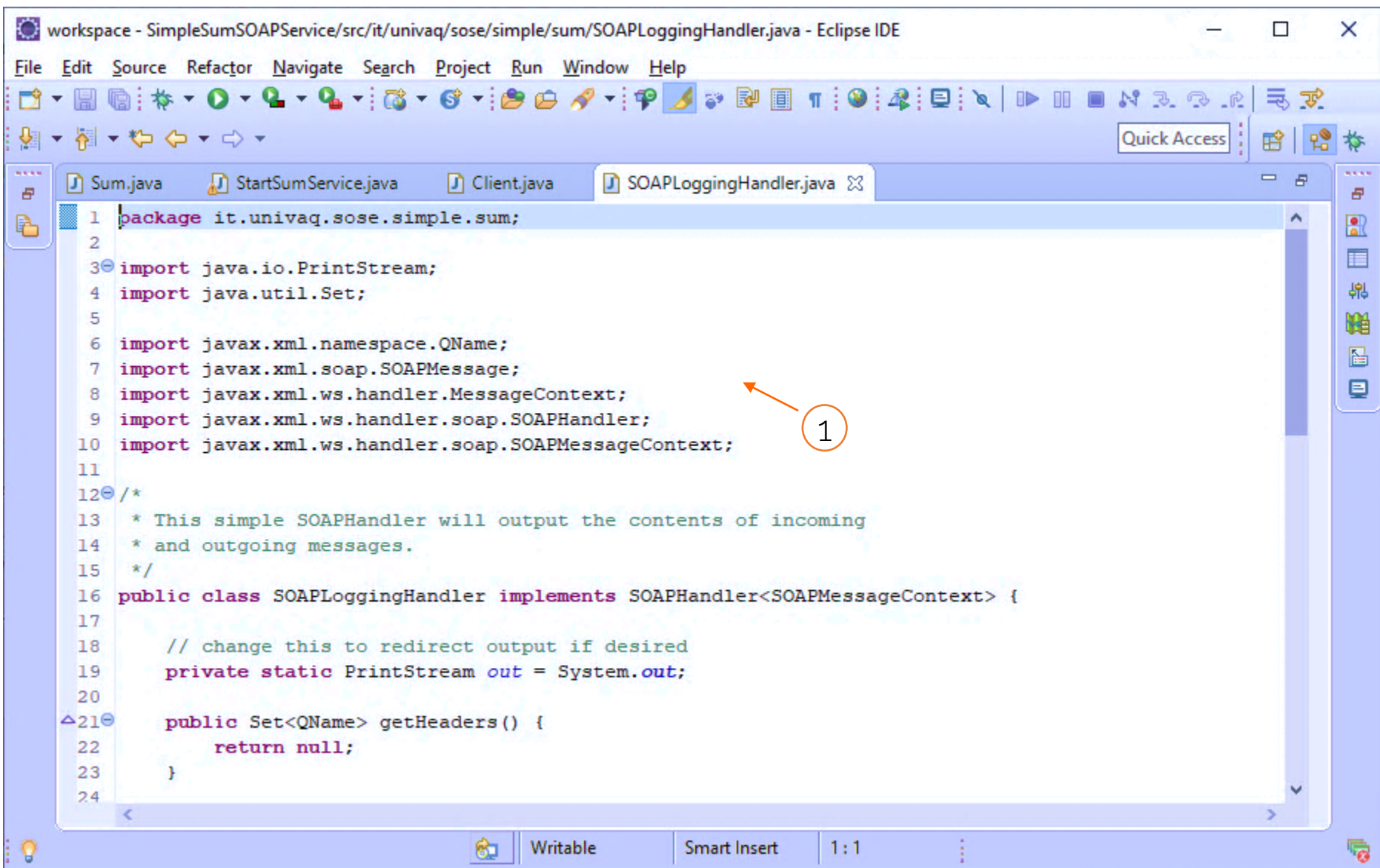


https://docs.oracle.com/cd/E13222_01/wls/docs103/webserv_adv/handlers.html

Order of Execution of Handler Methods



Create a SOAPLoggingHandler Class

The screenshot shows the Eclipse IDE interface. The title bar reads "workspace - SimpleSumSOAPService/src/it/univaq/sose/simple/sum/SOAPLoggingHandler.java - Eclipse IDE". The menu bar includes File, Edit, Source, Refactor, Navigate, Search, Project, Run, Window, and Help. The toolbar contains various icons for file operations, editing, and running. The package explorer on the left shows the project structure with files Sum.java, StartSumService.java, Client.java, and SOAPLoggingHandler.java. The editor window displays the code for SOAPLoggingHandler.java. The code includes package and import statements, a comment, and the start of a class that implements SOAPHandler. An orange arrow points from a circled number "1" to the import statement for javax.xml.ws.handler.soap.SOAPHandler.

```
1 package it.univaq.sose.simple.sum;
2
3 import java.io.PrintStream;
4 import java.util.Set;
5
6 import javax.xml.namespace.QName;
7 import javax.xml.soap.SOAPMessage;
8 import javax.xml.ws.handler.MessageContext;
9 import javax.xml.ws.handler.soap.SOAPHandler;
10 import javax.xml.ws.handler.soap.SOAPMessageContext;
11
12 /*
13  * This simple SOAPHandler will output the contents of incoming
14  * and outgoing messages.
15  */
16 public class SOAPLoggingHandler implements SOAPHandler<SOAPMessageContext> {
17
18     // change this to redirect output if desired
19     private static PrintStream out = System.out;
20
21     public Set<QName> getHeaders() {
22         return null;
23     }
24 }
```

Create a SOAPLoggingHandler Class

workspace - SimpleSumSOAPService/src/it/univaq/sose/simple/sum/SOAPLoggingHandler.java - Eclipse IDE

File Edit Source Refactor Navigate Search Project Run Window Help

Sum.java StartSumService.java Client.java SOAPLoggingHandler.java

```
16 public class SOAPLoggingHandler implements SOAPHandler<SOAPMessageContext> {
17
18     // change this to redirect output if desired
19     private static PrintStream out = System.out;
20
21     public Set<QName> getHeaders() {
22         return null;
23     }
24
25     public boolean handleMessage(SOAPMessageContext smc) {
26         logToSystemOut(smc);
27         return true;
28     }
29
30     public boolean handleFault(SOAPMessageContext smc) {
31         logToSystemOut(smc);
32         return true;
33     }
34
35     // nothing to clean up
36     public void close(MessageContext messageContext) {
37     }
38 }
```

2

3

4

5

6

```
classDiagram
    class Handler["Handler<T>"] {
        T extends MessageContext
        init(Map<String, Object>) void
        destroy() void
        handleMessage(T) boolean
        handleFault(T) boolean
        close(MessageContext) void
    }
    class LogicalHandler["LogicalHandler<T>"] {
        T extends LogicalMessageContext
    }
    class SOAPHandler["SOAPHandler<T>"] {
        T extends SOAPMessageContext
        getHeaders() Set<QName>
    }
    Handler <|-- LogicalHandler
    Handler <|-- SOAPHandler
```


Create a SOAPLoggingHandler Class

```
workspace - SimpleSumSOAPService/src/it/univaq/sose/simple/sum/SOAPLoggingHandler.java - Eclipse IDE
File Edit Source Refactor Navigate Search Project Run Window Help
Sum.java StartSumService.java Client.java SOAPLoggingHandler.java
39  /*
40  * Check the MESSAGE_OUTBOUND_PROPERTY in the context to see if this is an
41  * outgoing or incoming message. Write a brief message to the print stream and
42  * output the message. The writeTo() method can throw SOAPException or
43  * IOException
44  */
45  private void logToSystemOut(SOAPMessageContext smc) {
46      Boolean outboundProperty = (Boolean) smc.get(MessageContext.MESSAGE_OUTBOUND_PROPERTY);
47
48      if (outboundProperty.booleanValue()) {
49          out.println("\nOutbound message:");
50      } else {
51          out.println("\nInbound message:");
52      }
53
54      SOAPMessage message = smc.getMessage();
55      try {
56          message.writeTo(out);
57          out.println(""); // just to add a newline
58      } catch (Exception e) {
59          out.println("Exception in handler: " + e);
60      }
61  }
62  }
```

Register the SOAPLoggingHandler

workspace - SimpleSumSOAPService/src/it/univaq/sose/simple/sum/StartSumService.java - Eclipse IDE

File Edit Source Refactor Navigate Search Project Run Window Help

Sum.java StartSumService.java Client.java SOAPLoggingHandler.java

```
1 package it.univaq.ose.simple.sum;
2
3 import java.util.List;
4
5 import javax.xml.ws.Endpoint;
6 import javax.xml.ws.handler.Handler;
7
8 public class StartSumService {
9
10     public static void main(String[] args) {
11         Endpoint ep = Endpoint.create(new SumImpl());
12         List<Handler> handlerChain = ep.getBinding().getHandlerChain();
13         handlerChain.add(new SOAPLoggingHandler());
14         ep.getBinding().setHandlerChain(handlerChain);
15         ep.publish("http://localhost:8087/sum");
16     }
17 }
18
19
20
```

1

2

Execute the Service, Client ... check the Console

workspace - SimpleSumSOAPService/src/it/univaq/sose/simple/sum/StartSumService.java - Eclipse IDE

File Edit Source Refactor Navigate Search Project Run Window Help

Markers Properties Servers Data Source Explorer Snippets Console

StartSumService [Java Application] C:\Program Files\Java\jre1.8.0_221\bin\javaw.exe (24 set 2019, 15:42:51)

1 Inbound message:

```
<S:Envelope xmlns:S="http://schemas.xmlsoap.org/soap/envelope/" xmlns:SOAP-ENV="http://schemas.xmlsoap.org/soap/envelope/">
  <SOAP-ENV:Header />
  <S:Body>
    <ns2:sum xmlns:ns2="http://sum.simple.sose.univaq.it/">
      <arg0>5</arg0>
      <arg1>5</arg1>
    </ns2:sum>
  </S:Body>
</S:Envelope>
```

Outbound message:

```
<S:Envelope xmlns:S="http://schemas.xmlsoap.org/soap/envelope/" xmlns:SOAP-ENV="http://schemas.xmlsoap.org/soap/envelope/">
  <SOAP-ENV:Header />
  <S:Body>
    <ns2:sumResponse xmlns:ns2="http://sum.simple.sose.univaq.it/">
      <return>10</return>
    </ns2:sumResponse>
  </S:Body>
</S:Envelope>
```

2

TIP: Use the icon on the right to switch between the Server and the Client Console

20/03/2026

71

So far so good...

Let us modify our service a bit by adding

- a new sum operation

```
public Result sumBis(InputParam inputParam)
```

- and a multiplication operation

```
public Result mult(InputParam inputParam)
```

both take objects as input and return objects as output, rather than a simple type, i.e., rather than simply **int**

Create the **InputParam** and **Result** classes

InputParam.java Result.java

```
1 package it.univaq.sose.simple.sum;
2
3 public class InputParam {
4
5     private int a, b;
6
7     public InputParam() {
8         super();
9     }
10
11     public InputParam(int a, int b) {
12         this.a = a;
13         this.b = b;
14     }
15
16     public int getA() {
17         return a;
18     }
19
20     public void setA(int a) {
21         this.a = a;
22     }
23
24     public int getB() {
25         return b;
26     }
27
28     public void setB(int b) {
29         this.b = b;
30     }
31
32 }
```

These are the default constructors; without them you will get an error at runtime. Then, try yourself...

InputParam.java Result.java

```
1 package it.univaq.sose.simple.sum;
2
3 public class Result {
4
5     private int res;
6
7     public Result() {
8         super();
9     }
10
11     public Result(int res) {
12         this.res = res;
13     }
14
15     public int getRes() {
16         return res;
17     }
18
19     public void setRes(int res) {
20         this.res = res;
21     }
22
23 }
24
```

Modify the **sum** interface and its **implementation**

```
Sum.java SumImpl.java
1 package it.univaq.sose.simple.sum;
2
3 import javax.jws.WebService;
4
5 @WebService(endpointInterface = "it.univaq.sose.simple.sum.Sum")
6 public class SumImpl implements Sum {
7
8     public int sum(int a, int b) {
9         return a + b;
10    }
11
12    @Override
13    public Result sumBis(InputParam inputParam) {
14
15        return new Result(inputParam.getA() + inputParam.getB());
16    }
17
18    @Override
19    public Result mult(InputParam inputParam) {
20        return new Result(inputParam.getA() * inputParam.getB());
21    }
22 }
23
24
```

```
Sum.java SumImpl.java
1 package it.univaq.sose.simple.sum;
2
3 import javax.jws.WebMethod;
4 import javax.jws.WebService;
5
6 @WebService
7 public interface Sum {
8
9     @WebMethod
10    public int sum(int a, int b);
11
12    @WebMethod
13    public Result sumBis(InputParam inputParam);
14
15    @WebMethod
16    public Result mult(InputParam inputParam);
17 }
18
19
```

Once again, simply run the `StartSumService`

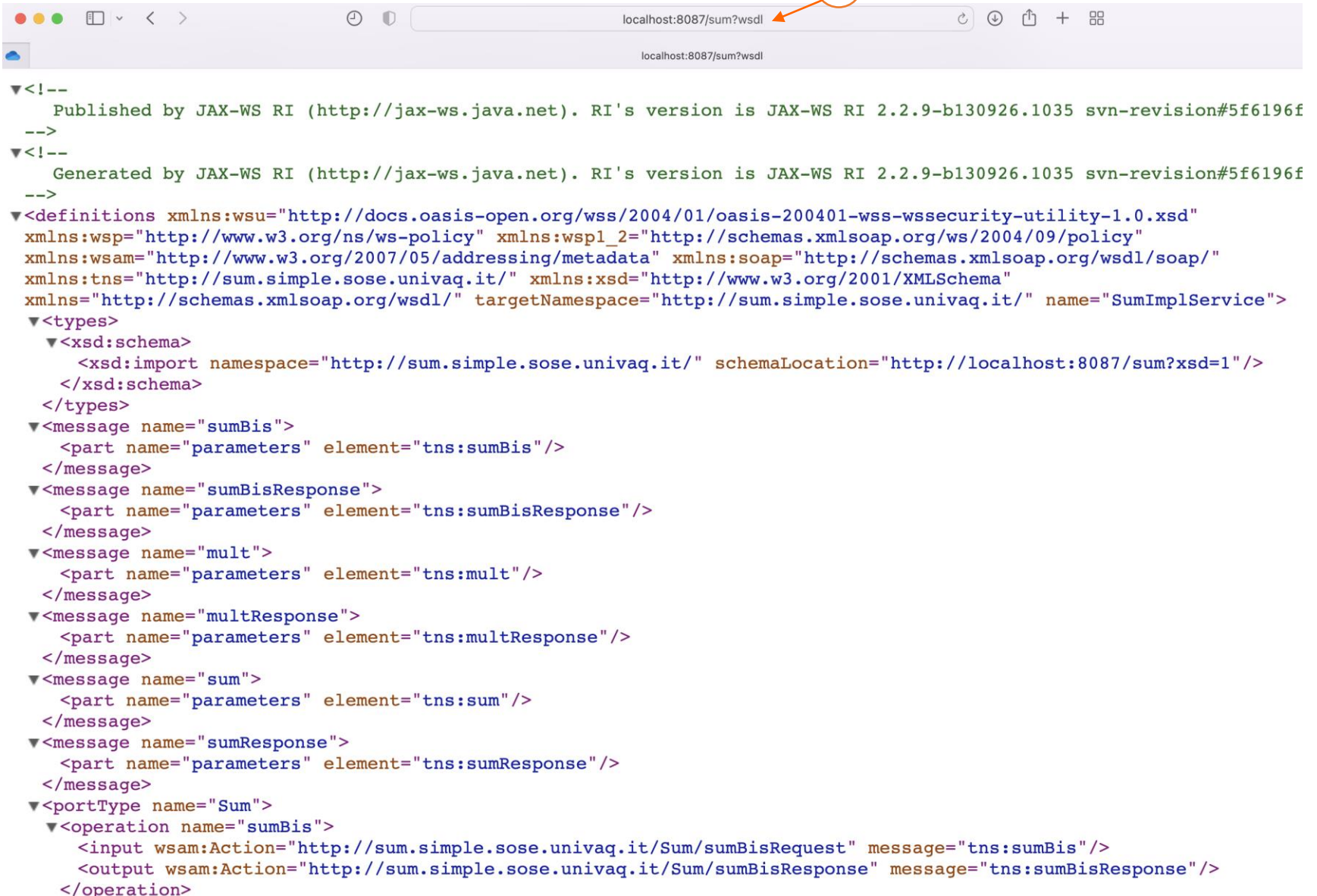
The screenshot shows an IDE with the following components:

- Project Explorer (Left):** Displays a project structure. The 'src' folder is expanded, showing the package 'it.univaq.sose.simple.sum'. The file 'StartSumService.java' is selected.
- Code Editor (Center):** Shows the content of 'StartSumService.java':


```
1 package it.univaq.sose.simple.sum;
2
3 import java.util.List;
4
5 public class StartSumService {
6
7     public static void main(String[] args) {
8
9         // Endpoint.publish("http://localhost:8087/sur
10
11         Endpoint ep = Endpoint.create(new Sur
12         List<Handler> handlerChain = ep.getB
13         handlerChain.add(new SOAPLoggingHand
14         ep.getBinding().setHandlerChain(hand
15         ep.publish("http://localhost:8087/sur
16
17     }
18
19 }
20
21 }
22
23 }
```
- Run Menu (Bottom Right):** A context menu is open, showing options like 'Run As' and 'Debug As'. A red circle with the number '1' is drawn around the 'Run As' option.

Once again, the Service is up (WSDL 1/2)

1

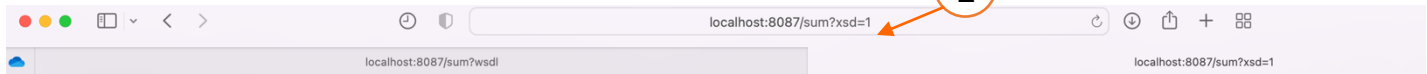


```
<!--
  Published by JAX-WS RI (http://jax-ws.java.net). RI's version is JAX-WS RI 2.2.9-b130926.1035 svn-revision#5f6196f
-->
<!--
  Generated by JAX-WS RI (http://jax-ws.java.net). RI's version is JAX-WS RI 2.2.9-b130926.1035 svn-revision#5f6196f
-->
<definitions xmlns:wsu="http://docs.oasis-open.org/wss/2004/01/oasis-200401-wss-wssecurity-utility-1.0.xsd"
  xmlns:wsp="http://www.w3.org/ns/ws-policy" xmlns:wsp1_2="http://schemas.xmlsoap.org/ws/2004/09/policy"
  xmlns:wsam="http://www.w3.org/2007/05/addressing/metadata" xmlns:soap="http://schemas.xmlsoap.org/wsdl/soap/"
  xmlns:tns="http://sum.simple.sose.univaq.it/" xmlns:xsd="http://www.w3.org/2001/XMLSchema"
  xmlns="http://schemas.xmlsoap.org/wsdl/" targetNamespace="http://sum.simple.sose.univaq.it/" name="SumImplService">
  <types>
    <xsd:schema>
      <xsd:import namespace="http://sum.simple.sose.univaq.it/" schemaLocation="http://localhost:8087/sum?xsd=1"/>
    </xsd:schema>
  </types>
  <message name="sumBis">
    <part name="parameters" element="tns:sumBis"/>
  </message>
  <message name="sumBisResponse">
    <part name="parameters" element="tns:sumBisResponse"/>
  </message>
  <message name="mult">
    <part name="parameters" element="tns:mult"/>
  </message>
  <message name="multResponse">
    <part name="parameters" element="tns:multResponse"/>
  </message>
  <message name="sum">
    <part name="parameters" element="tns:sum"/>
  </message>
  <message name="sumResponse">
    <part name="parameters" element="tns:sumResponse"/>
  </message>
  <portType name="Sum">
    <operation name="sumBis">
      <input wsam:Action="http://sum.simple.sose.univaq.it/Sum/sumBisRequest" message="tns:sumBis"/>
      <output wsam:Action="http://sum.simple.sose.univaq.it/Sum/sumBisResponse" message="tns:sumBisResponse"/>
    </operation>
```

Once again, the Service is up (WSDL 2/2)

```
▼<operation name="mult">
  <input wsam:Action="http://sum.simple.sose.univaq.it/Sum/multRequest" message="tns:mult"/>
  <output wsam:Action="http://sum.simple.sose.univaq.it/Sum/multResponse" message="tns:multResponse"/>
</operation>
▼<operation name="sum">
  <input wsam:Action="http://sum.simple.sose.univaq.it/Sum/sumRequest" message="tns:sum"/>
  <output wsam:Action="http://sum.simple.sose.univaq.it/Sum/sumResponse" message="tns:sumResponse"/>
</operation>
</portType>
▼<binding name="SumImplPortBinding" type="tns:Sum">
  <soap:binding transport="http://schemas.xmlsoap.org/soap/http" style="document"/>
  ▼<operation name="sumBis">
    <soap:operation soapAction=""/>
    ▼<input>
      <soap:body use="literal"/>
    </input>
    ▼<output>
      <soap:body use="literal"/>
    </output>
  </operation>
  ▼<operation name="mult">
    <soap:operation soapAction=""/>
    ▼<input>
      <soap:body use="literal"/>
    </input>
    ▼<output>
      <soap:body use="literal"/>
    </output>
  </operation>
  ▼<operation name="sum">
    <soap:operation soapAction=""/>
    ▼<input>
      <soap:body use="literal"/>
    </input>
    ▼<output>
      <soap:body use="literal"/>
    </output>
  </operation>
</binding>
▼<service name="SumImplService">
  ▼<port name="SumImplPort" binding="tns:SumImplPortBinding">
    <soap:address location="http://localhost:8087/sum"/>
  </port>
</service>
</definitions>
```

The Service is up (XSD)



```
<?xml version='1.0'>
<xs:schema xmlns:tns="http://sum.simple.sose.univaq.it/" xmlns:xs="http://www.w3.org/2001/XMLSchema" version="1.0"
targetNamespace="http://sum.simple.sose.univaq.it/">
  <xs:element name="mult" type="tns:mult"/>
  <xs:element name="multResponse" type="tns:multResponse"/>
  <xs:element name="sum" type="tns:sum"/>
  <xs:element name="sumBis" type="tns:sumBis"/>
  <xs:element name="sumBisResponse" type="tns:sumBisResponse"/>
  <xs:element name="sumResponse" type="tns:sumResponse"/>
  <xs:complexType name="sum">
    <xs:sequence>
      <xs:element name="arg0" type="xs:int"/>
      <xs:element name="arg1" type="xs:int"/>
    </xs:sequence>
  </xs:complexType>
  <xs:complexType name="sumResponse">
    <xs:sequence>
      <xs:element name="return" type="xs:int"/>
    </xs:sequence>
  </xs:complexType>
  <xs:complexType name="sumBis">
    <xs:sequence>
      <xs:element name="arg0" type="tns:inputParam" minOccurs="0"/>
    </xs:sequence>
  </xs:complexType>
  <xs:complexType name="inputParam">
    <xs:sequence>
      <xs:element name="a" type="xs:int"/>
      <xs:element name="b" type="xs:int"/>
    </xs:sequence>
  </xs:complexType>
  <xs:complexType name="sumBisResponse">
    <xs:sequence>
      <xs:element name="return" type="tns:result" minOccurs="0"/>
    </xs:sequence>
  </xs:complexType>
  <xs:complexType name="result">
    <xs:sequence>
      <xs:element name="res" type="xs:int"/>
    </xs:sequence>
  </xs:complexType>
  <xs:complexType name="mult">
    <xs:sequence>
      <xs:element name="arg0" type="tns:inputParam" minOccurs="0"/>
    </xs:sequence>
  </xs:complexType>
  <xs:complexType name="multResponse">
    <xs:sequence>
      <xs:element name="return" type="tns:result" minOccurs="0"/>
    </xs:sequence>
  </xs:complexType>
</xs:schema>
```


Modify the Client and run it

```
Client.java
1 package it.univaq.sose.simple.sum.client;
2
3 import java.net.MalformedURLException;
4 import java.net.URL;
5
6 import javax.xml.namespace.QName;
7 import javax.xml.ws.Service;
8
9 import it.univaq.sose.simple.sum.InputParam;
10 import it.univaq.sose.simple.sum.Result;
11 import it.univaq.sose.simple.sum.Sum;
12
13 public class Client {
14
15     public static void main(String[] args) {
16
17         try {
18             URL wsdlURL = new URL("http://localhost:8087/sum?wsdl");
19             QName SERVICE_NAME = new QName("http://sum.simple.sose.univaq.it/", "SumImplService");
20             QName PORT_NAME = new QName("http://sum.simple.sose.univaq.it/", "SumImplPort");
21
22             Service service = Service.create(wsdlURL, SERVICE_NAME);
23
24             Sum port = service.getPort(PORT_NAME, Sum.class);
25
26             int result = port.sum(5, 5);
27             System.out.println("The result of \"port.sum(5, 5)\" is : " + result);
28
29             Result objResult = port.sumBis(new InputParam(4, 8));
30             System.out.println("The result of \"port.sumBis(new InputParam(4, 8))\" is : " + objResult.getRes());
31
32             objResult = port.mult(new InputParam(4, 8));
33             System.out.println("The result of \"port.mult(new InputParam(4, 8))\" is : " + objResult.getRes());
34
35         } catch (MalformedURLException e) {
36             e.printStackTrace();
37         }
38     }
39 }
```

Once again, so far so good... output 1/3

Markers Properties Servers Data Source Explorer Snippets Console JUnit
<terminated> Client [Java Application] /Library/Internet Plug-Ins/JavaAppletPlugin.plugin/Contents/Home/bin/java (Mar 8, 2022, 12:30)

The result of "port.sum(5, 5)" is : 10
The result of "port.sumBis(new InputParam(4, 8))" is : 12
The result of "port.mult(new InputParam(4, 8))" is : 32

Formatted XML

```
1 Inbound message:
2
3 <S:Envelope
4   xmlns:S="http://schemas.xmlsoap.org/soap/envelope/"
5   xmlns:SOAP-ENV="http://schemas.xmlsoap.org/soap/envelope/">
6   <SOAP-ENV:Header/>
7   <S:Body>
8     <ns2:sum
9       xmlns:ns2="http://sum.simple.sose.univaq.it/"
10      <arg0>5</arg0>
11      <arg1>5</arg1>
12    </ns2:sum>
13  </S:Body>
14 </S:Envelope>
15
16 Outbound message:
17
18 <S:Envelope
19   xmlns:S="http://schemas.xmlsoap.org/soap/envelope/"
20   xmlns:SOAP-ENV="http://schemas.xmlsoap.org/soap/envelope/">
21   <SOAP-ENV:Header/>
22   <S:Body>
23     <ns2:sumResponse
24       xmlns:ns2="http://sum.simple.sose.univaq.it/"
25       <return>10</return>
26     </ns2:sumResponse>
27   </S:Body>
28 </S:Envelope>
29
```

Once again, so far so good... output 2/3

Markers Properties Servers Data Source Explorer Snippets Console JUnit
<terminated> Client [Java Application] /Library/Internet Plug-Ins/JavaAppletPlugin.plugin/Contents/Home/bin/java (Mar 8, 2022, 12:30)

The result of "port.sum(5, 5)" is : 10

The result of "port.sumBis(new InputParam(4, 8))" is : 12 ←

The result of "port.mult(new InputParam(4, 8))" is : 32

```
Formatted XML
29
30 Inbound message:
31
32 <S:Envelope
33   xmlns:S="http://schemas.xmlsoap.org/soap/envelope/"
34   xmlns:SOAP-ENV="http://schemas.xmlsoap.org/soap/envelope/"
35   <SOAP-ENV:Header/>
36   <S:Body>
37     <ns2:sumBis
38       xmlns:ns2="http://sum.simple.sose.univaq.it/"
39       <arg0>
40         <a>4</a>
41         <b>8</b>
42       </arg0>
43     </ns2:sumBis>
44   </S:Body>
45 </S:Envelope>
46
47 Outbound message:
48
49 <S:Envelope
50   xmlns:S="http://schemas.xmlsoap.org/soap/envelope/"
51   xmlns:SOAP-ENV="http://schemas.xmlsoap.org/soap/envelope/"
52   <SOAP-ENV:Header/>
53   <S:Body>
54     <ns2:sumBisResponse
55       xmlns:ns2="http://sum.simple.sose.univaq.it/"
56       <return>
57         <res>12</res>
58       </return>
59     </ns2:sumBisResponse>
60   </S:Body>
61 </S:Envelope>
62
```

Once again, so far so good... output 3/3

Markers Properties Servers Data Source Explorer Snippets Console JUnit
<terminated> Client [Java Application] /Library/Internet Plug-Ins/JavaAppletPlugin.plugin/Contents/Home/bin/java (Mar 8, 2022, 12:30)

The result of "port.sum(5, 5)" is : 10

The result of "port.sumBis(new InputParam(4, 8))" is : 12

The result of "port.mult(new InputParam(4, 8))" is : 32 ←

Everything looks great!

So, what are we missing? ...

Formatted XML

```
62
63 Inbound message:
64
65 <S:Envelope
66   xmlns:S="http://schemas.xmlsoap.org/soap/envelope/"
67   xmlns:SOAP-ENV="http://schemas.xmlsoap.org/soap/envelope/">
68   <SOAP-ENV:Header/>
69   <S:Body>
70     <ns2:mult
71       xmlns:ns2="http://sum.simple.sose.univaq.it/">
72       <arg0>
73         <a>4</a>
74         <b>8</b>
75       </arg0>
76     </ns2:mult>
77   </S:Body>
78 </S:Envelope>
79
80 Outbound message:
81
82 <S:Envelope
83   xmlns:S="http://schemas.xmlsoap.org/soap/envelope/"
84   xmlns:SOAP-ENV="http://schemas.xmlsoap.org/soap/envelope/">
85   <SOAP-ENV:Header/>
86   <S:Body>
87     <ns2:multResponse
88       xmlns:ns2="http://sum.simple.sose.univaq.it/">
89       <return>
90         <res>32</res>
91       </return>
92     </ns2:multResponse>
93   </S:Body>
94 </S:Envelope>
```

Once again, so far so good... output 3/3

Markers Properties Servers Data Source Explorer Snippets Console JUnit
<terminated> Client [Java Application] /Library/Internet Plug-Ins/JavaAppletPlugin.plugin/Contents/Home/bin/java (Mar 8, 2022, 12:30)

The result of "port.sum(5, 5)" is : 10

The result of "port.sumBis(new InputParam(4, 8))" is : 12

The result of "port.mult(new InputParam(4, 8))" is : 32 ←

We have not used the WSDL at all ☹

We have interacted with the server directly by writing the Java code of the Client:

- "in principle", that would be perfectly fine... the problem is that writing this code has required the knowledge of the data structures used by the server side (basically, we know implementation details of the server ☹ ... next slide)

Formatted XML

```
62
63 Inbound message:
64
65 <S:Envelope
66   xmlns:S="http://schemas.xmlsoap.org/soap/envelope/"
67   xmlns:SOAP-ENV="http://schemas.xmlsoap.org/soap/envelope/">
68   <SOAP-ENV:Header/>
69   <S:Body>
70     <ns2:mult
71       xmlns:ns2="http://sum.simple.sose.univaq.it/">
72       <arg0>
73         <a>4</a>
74         <b>8</b>
75       </arg0>
76     </ns2:mult>
77   </S:Body>
78 </S:Envelope>
79
80 Outbound message:
81
82 <S:Envelope
83   xmlns:S="http://schemas.xmlsoap.org/soap/envelope/"
84   xmlns:SOAP-ENV="http://schemas.xmlsoap.org/soap/envelope/">
85   <SOAP-ENV:Header/>
86   <S:Body>
87     <ns2:multResponse
88       xmlns:ns2="http://sum.simple.sose.univaq.it/">
89       <return>
90         <res>32</res>
91       </return>
92     </ns2:multResponse>
93   </S:Body>
94 </S:Envelope>
```

Let's have a closer look at the Client code

```
Client.java
1 package it.univaq.sose.simple.sum.client;
2
3 import java.net.MalformedURLException;
4 import java.net.URL;
5
6 import javax.xml.namespace.QName;
7 import javax.xml.ws.Service;
8
9 import it.univaq.sose.simple.sum.InputParam;
10 import it.univaq.sose.simple.sum.Result;
11 import it.univaq.sose.simple.sum.Sum;
12
13 public class Client {
14
15     public static void main(String[] args) {
16
17         try {
18             URL wsdlURL = new URL("http://localhost:8087/sum?wsdl");
19             QName SERVICE_NAME = new QName("http://sum.simple.sose.univaq.it/", "SumImplService");
20             QName PORT_NAME = new QName("http://sum.simple.sose.univaq.it/", "SumImplPort");
21
22             Service service = Service.create(wsdlURL, SERVICE_NAME);
23
24             Sum port = service.getPort(PORT_NAME, Sum.class);
25
26             int result = port.sum(5, 5);
27             System.out.println("The result of \"port.sum(5, 5)\" is : " + result);
28
29             Result objResult = port.sumBis(new InputParam(4, 8));
30             System.out.println("The result of \"port.sumBis(new InputParam(4, 8))\" is : " + objResult.getRes());
31
32             objResult = port.mult(new InputParam(4, 8));
33             System.out.println("The result of \"port.mult(new InputParam(4, 8))\" is : " + objResult.getRes());
34
35         } catch (MalformedURLException e) {
36             e.printStackTrace();
37         }
38     }
39 }
```

Note that in the Client side we use the classes InputParam and Result... The very same classes used by the Server side



What's the take-away message?

- Did we realize our service and client by adopting **SOA best practices**?
 - What about the concept that a service is an abstract entity that is implementation neutral?
 - We should have spent more time working out the exact interface/contract our service requires before start coding 
 - We did not use the WSDL at all, i.e., we did not use the service contract at all
 - We directly wrote Java code, on both the server and client side 
- Have we met **SOA principles**?
 - As for the **Service Abstraction principle**, we did not hide the logic our service encapsulates from the outside world 
 - We had to deal with implementation details such as the programming language (Java, in our case) and the specific input/output data types. On the client side, we directly used classes such as **InputParam** and **Result**, even importing the same class definitions from the server-side package. This clearly violates service abstraction and loose coupling principles, resulting in tight coupling between client and service.
- This is **not service orientation**; this is **shared code disguised as a service**. 

What's the take-away message?

Today

- REST dominates modern service development
- SOAP survives in enterprise, banking, and government contexts

From an academic perspective, SOAP remains very useful because it:

- illustrates formal service contracts (WSDL)
- introduces schema-driven design
- highlights the distinction between service contract and implementation

SOAP is well suited for explaining foundational SOA principles

From legacy model to modularity

Starting from Java 11, JAX-WS (and the Endpoint class) was removed from the JDK, requiring external libraries or application servers to develop SOAP services

- Java 9 → deprecated Java 11 → removed

SOAP in Java today is no longer “built-in”, while REST is typically implemented using frameworks such as Spring Boot

Oracle removed the following from the JDK:

- JAX-WS, JAXB, CORBA

Reasons:

- to reduce the size of the JDK
- to separate enterprise technologies from the core platform
- to promote modularity and the use of external libraries

Transition from Java “batteries included” (legacy model) to modular Java + an ecosystem of external libraries (modern model)

From legacy model to modularity

Today, there are two main options:

Add external libraries (standalone)

- For example: Apache CXF or Metro (JAX-WS RI).
- In this case, you can still use `Endpoint.publish(...)`.

Use a container or application server

- For example: Tomcat, WildFly, or Spring Boot.
- In this case, you do not use `Endpoint` directly; instead, you deploy the service within the container

Take-home message

- As software engineers, you will not always work with the latest technologies
- More often, you will maintain, migrate, debug, and evolve existing systems
- It is essential to understand where they come from and how they work

Although working only with the latest technologies may seem easier and more appealing, real-world engineering often requires dealing with existing systems, and that is where real value is created

References

- <https://www.eclipse.org/downloads/packages>
- <https://docs.oracle.com/javaee/6/api/javax/xml/ws/Service.html>
- <http://download.oracle.com/javaee/5/tutorial/doc/bnazq.html#bnazs>
- https://jax-ws.java.net/articles/handlers_introduction.html
- <https://stackoverflow.com/questions/1945618/tracing-xml-request-responses-with-jax-ws>
- https://docs.oracle.com/cd/E13222_01/wls/docs103/webserv_adv/handlers.html